

University of Moratuwa  
Faculty of Engineering / Department of Electronic and  
Telecommunication Engineering

---

**Design, Development and FPGA  
Implementation of a Complete GMSK  
Modem using High-Level Synthesis**

---

**Author:** Dulina Hansaja Ranaweera  
**Supervisor:** Mr. Sanjith Shanmugathashan  
**Host Organization:** Thakshana Technologies Private Limited

July 2026

# Abstract

Gaussian Minimum Shift Keying (GMSK) remains one of the most important constant-envelope modulation schemes in digital wireless communication, most prominently as the physical-layer modulation of the GSM cellular standard. This report documents the complete, independently executed engineering development of a GMSK burst modem, beginning from communication-theoretic first principles and culminating in a hardware implementation of its channel-coding subsystem on a Xilinx Zynq-7000 FPGA using High-Level Synthesis (HLS). The work proceeded in clearly identifiable stages. A complete floating-point MATLAB model of the modem was first constructed, comprising a rate- $1/2$ , constraint-length-7 convolutional encoder with generator polynomials  $(171, 133)_8$ , a burst-oriented framing structure with preamble and interleaved training fields, Gaussian pulse shaping with  $BT = 0.35$ , a lookup-table-based GMSK phase mapper, a phase-derivative demodulator producing soft log-likelihood metrics, correlation-based preamble synchronization, and a sliding-window soft-decision Viterbi decoder. The model was validated end-to-end under additive white Gaussian noise (AWGN), and a statistically rigorous bit-error-rate (BER) evaluation methodology was developed, targeting a BER of  $10^{-5}$ . The evaluation environment was then extended from the ideal AWGN channel to standardized mobile radio channels. Following an investigation of the COST 207 propagation models adopted by GSM, the severe `gsmHTx12c1` hilly-terrain profile with a 200 Hz Doppler spread was integrated into the simulation. The immediate and total failure of the AWGN-designed receiver under this channel ( $\text{BER} = 1$ ) motivated a systematic receiver enhancement programme: least-squares training-based channel estimation, complex-domain normalized least-mean-squares (NLMS) adaptive equalization, decision-delay optimization, and, after an extended root-cause investigation of burst-timing and reference-signal defects, a preamble-trained equalizer architecture that achieved error-free payload recovery over the hilly-terrain channel and reached the target BER of  $10^{-5}$  at approximately 24 dB SNR. In parallel, the convolutional encoder and a 64-state trace-back Viterbi decoder were implemented in Vitis HLS with AXI4-Stream interfaces, verified against golden software references, debugged through C and RTL co-simulation, and integrated into a self-contained hardware throughput measurement system driven by an on-chip Galois LFSR and instrumented with an Integrated Logic Analyzer. The measured decoded throughput of the encoder–decoder chain was approximately 32.65 Mb/s on the

Zybo Z7-10 board, in close agreement with RTL simulation (32.7 Mb/s at 100 MHz), with the Viterbi decoder identified as the throughput bottleneck at an effective initiation interval of approximately 49–51 cycles. The report documents not only the final system but the complete engineering journey, including failed approaches, design iterations, debugging investigations and the reasoning behind every major architectural decision, and concludes with a critical evaluation of the results against theoretical expectations and a roadmap for future work.

# Acknowledgements

I would like to express my sincere gratitude to Mr. Nithiananthan Nitkunanantha, Director of Thakshana Technologies (PVT) Limited, who conducted my interview and served as my immediate supervisor. His vision for DSP and FPGA engineering provided constant inspiration throughout the internship.

I am equally grateful to Ms. Bhashini Jayathunga, Senior Embedded and FPGA Engineer, who was the second interviewer and whose technical expertise guided my initial understanding of FPGA design flows and embedded system integration.

My deepest thanks go to Mr. Sanjith Shanmugathashan, Senior DSP Engineer and direct supervisor, whose patient mentorship and detailed technical feedback were indispensable for the successful completion of the GMSK modem project. His guidance in MATLAB-based DSP modelling, HLS implementation, and FPGA debugging shaped the core of my learning experience.

I would also like to thank Ms. Janani Wimalarathne, Project Manager, for coordinating project milestones and ensuring a structured and productive internship environment, and Ms. Aysha Aniff, HR Manager, for facilitating all administrative aspects of the internship with professionalism and care.

I am grateful to my fellow intern and batch mate Mr. Mathujan Shanmugavadivel for his collaboration, technical discussions, and camaraderie throughout the training period.

Finally, I thank the University of Moratuwa and the Department of Electronic and Telecommunication Engineering for providing this invaluable industrial training opportunity as part of the undergraduate curriculum.

# Declaration

I declare that this report and the work described within it are my own, carried out independently as part of my internship work, except where explicitly stated otherwise. All sources of information have been acknowledged by means of references.

Signed: 

Date: 4th of July 2026

# Contents

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                    | <b>1</b>  |
| <b>Acknowledgements</b>                                            | <b>3</b>  |
| <b>Declaration</b>                                                 | <b>4</b>  |
| <b>List of Figures</b>                                             | <b>9</b>  |
| <b>List of Tables</b>                                              | <b>10</b> |
| <b>Abbreviations</b>                                               | <b>11</b> |
| <b>1 Introduction</b>                                              | <b>13</b> |
| 1.1 Background . . . . .                                           | 13        |
| 1.2 Problem Statement . . . . .                                    | 14        |
| 1.3 Motivation . . . . .                                           | 14        |
| 1.4 Research Objectives . . . . .                                  | 15        |
| 1.5 Project Scope . . . . .                                        | 15        |
| 1.6 Expected Contributions . . . . .                               | 16        |
| 1.7 Report Structure and Chapter Summary . . . . .                 | 16        |
| <b>2 Literature Review</b>                                         | <b>17</b> |
| 2.1 Digital Communication Systems . . . . .                        | 17        |
| 2.2 Continuous Phase Modulation and Minimum Shift Keying . . . . . | 17        |
| 2.3 Gaussian Minimum Shift Keying . . . . .                        | 18        |
| 2.4 Convolutional Coding . . . . .                                 | 18        |
| 2.5 The Viterbi Algorithm . . . . .                                | 18        |
| 2.6 Soft-Decision Decoding . . . . .                               | 19        |
| 2.7 Mobile Radio Channels . . . . .                                | 19        |
| 2.8 FPGA Implementation of Communication Systems . . . . .         | 20        |
| 2.9 High-Level Synthesis . . . . .                                 | 20        |
| 2.10 Existing Implementations . . . . .                            | 20        |
| 2.11 Research Gaps Addressed . . . . .                             | 21        |

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| 2.12     | Chapter Summary . . . . .                                     | 21        |
| <b>3</b> | <b>System Requirements and Design Methodology</b>             | <b>22</b> |
| 3.1      | Overall Modem Architecture . . . . .                          | 22        |
| 3.2      | Requirements . . . . .                                        | 22        |
| 3.3      | Development Methodology . . . . .                             | 23        |
| 3.4      | Toolchain . . . . .                                           | 24        |
| 3.5      | Verification Flow . . . . .                                   | 24        |
| 3.6      | Engineering Workflow . . . . .                                | 25        |
| 3.6.1    | Organisational Context . . . . .                              | 25        |
| <b>4</b> | <b>System Design</b>                                          | <b>26</b> |
| 4.1      | Frame Structure . . . . .                                     | 26        |
| 4.2      | Convolutional Encoder . . . . .                               | 27        |
| 4.3      | Gaussian Filter . . . . .                                     | 27        |
| 4.4      | GMSK Modulator (Mapper) . . . . .                             | 28        |
| 4.5      | Channel Models . . . . .                                      | 28        |
| 4.5.1    | AWGN . . . . .                                                | 28        |
| 4.5.2    | Flat Rayleigh fading . . . . .                                | 28        |
| 4.5.3    | Standardized GSM multipath . . . . .                          | 29        |
| 4.6      | Synchronization . . . . .                                     | 29        |
| 4.7      | Demodulator . . . . .                                         | 30        |
| 4.8      | Viterbi Decoder Architecture . . . . .                        | 30        |
| 4.8.1    | Architecture selection . . . . .                              | 30        |
| 4.8.2    | Functional blocks . . . . .                                   | 31        |
| 4.8.3    | Block-mode versus sliding-window operation . . . . .          | 31        |
| 4.9      | Receiver Equalization Design . . . . .                        | 32        |
| 4.10     | Hardware Architecture and AXI4-Stream Interfaces . . . . .    | 33        |
| 4.10.1   | Interface requirement and the parallelization study . . . . . | 33        |
| 4.10.2   | Stream protocol design rules . . . . .                        | 34        |
| 4.11     | Signal Flow and Timing . . . . .                              | 34        |
| <b>5</b> | <b>Implementation</b>                                         | <b>35</b> |
| 5.1      | MATLAB Implementation . . . . .                               | 35        |
| 5.1.1    | Code organization . . . . .                                   | 35        |
| 5.1.2    | Legacy code analysis . . . . .                                | 35        |
| 5.1.3    | Soft-decision Viterbi reference decoder . . . . .             | 36        |
| 5.1.4    | Simulation drivers and performance engineering . . . . .      | 36        |
| 5.2      | HLS Implementation of the Convolutional Encoder . . . . .     | 36        |
| 5.3      | HLS Implementation of the Viterbi Decoder . . . . .           | 37        |

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| 5.3.1    | From studied RTL to HLS . . . . .                                     | 37        |
| 5.3.2    | The 64-state production decoder . . . . .                             | 37        |
| 5.3.3    | Parameterized and soft-decision variants . . . . .                    | 38        |
| 5.4      | Memory Organization . . . . .                                         | 38        |
| 5.5      | HLS Implementation of the GMSK Mapper . . . . .                       | 38        |
| 5.6      | On-Chip Measurement Infrastructure . . . . .                          | 39        |
| <b>6</b> | <b>Verification and Experimental Work</b>                             | <b>40</b> |
| 6.1      | MATLAB Functional Verification . . . . .                              | 40        |
| 6.2      | BER Methodology Development . . . . .                                 | 41        |
| 6.3      | Verification of the Soft-Decision Reference Decoder . . . . .         | 42        |
| 6.4      | Rayleigh-Stage Experiments . . . . .                                  | 42        |
| 6.5      | GSM Hilly-Terrain Integration and the BER = 1 Investigation . . . . . | 43        |
| 6.6      | Equalizer Development Experiments . . . . .                           | 43        |
| 6.7      | Realistic-Channel BER Campaign . . . . .                              | 44        |
| 6.8      | The Group-Delay Alignment Investigations . . . . .                    | 45        |
| 6.9      | HLS Encoder Verification . . . . .                                    | 45        |
| 6.10     | HLS Decoder Verification and the Block-Boundary Incident . . . . .    | 46        |
| 6.11     | RTL System Simulation: Loopback Testbenches . . . . .                 | 46        |
| 6.12     | Hardware Validation on the Zybo Z7-10 . . . . .                       | 48        |
| 6.13     | Validation Strategy Summary . . . . .                                 | 50        |
| <b>7</b> | <b>Results and Discussion</b>                                         | <b>51</b> |
| 7.1      | Communication Performance . . . . .                                   | 51        |
| 7.2      | Hardware Performance . . . . .                                        | 52        |
| 7.3      | Design Trade-offs Revisited . . . . .                                 | 53        |
| 7.4      | Comparison with Theoretical Expectations . . . . .                    | 53        |
| 7.5      | Engineering Observations . . . . .                                    | 54        |
| 7.6      | Limitations . . . . .                                                 | 54        |
| <b>8</b> | <b>Conclusion</b>                                                     | <b>55</b> |
| 8.1      | Achievements . . . . .                                                | 55        |
| 8.2      | Research Contributions . . . . .                                      | 55        |
| 8.3      | Lessons Learned . . . . .                                             | 56        |
| 8.4      | Future Work . . . . .                                                 | 56        |
| <b>A</b> | <b>MATLAB Source Files</b>                                            | <b>57</b> |
| <b>B</b> | <b>HLS Source Files</b>                                               | <b>59</b> |
| <b>C</b> | <b>RTL Diagrams</b>                                                   | <b>60</b> |

|          |                          |           |
|----------|--------------------------|-----------|
| <b>D</b> | <b>Simulation Setup</b>  | <b>61</b> |
| <b>E</b> | <b>Experimental Data</b> | <b>62</b> |
|          | <b>References</b>        | <b>64</b> |

# List of Figures

|     |                                                                                                                                                                      |    |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | Overall GMSK modem architecture and end-to-end signal flow. . . . .                                                                                                  | 22 |
| 4.1 | Burst structure: preamble, interleaved payload and training chunks . . . .                                                                                           | 27 |
| 4.2 | GMSK mapper datapath: shift, saturating accumulator, LUT index, sin/-<br>cos ROM . . . . .                                                                           | 28 |
| 4.3 | Viterbi decoder architecture: BMU, 64-way ACS array, survivor memory<br>banks, minimum-metric search, traceback unit . . . . .                                       | 32 |
| 4.4 | Final encoder hardware architecture: 32-bit AXI input register, bit-serial<br>core, 64-bit output buffer, two-word AXI output . . . . .                              | 34 |
| 6.1 | TX Power Spectral Density Plot . . . . .                                                                                                                             | 41 |
| 6.2 | AWGN BER versus SNR with $10^{-5}$ crossing annotation . . . . .                                                                                                     | 42 |
| 6.3 | flat Rayleigh without equalization . . . . .                                                                                                                         | 43 |
| 6.4 | GSM HT with NLMS equalization . . . . .                                                                                                                              | 44 |
| 6.5 | ILA capture on the Zybo Z7-10 showing the frozen counters <code>bits_q</code> and<br><code>cycs_q</code> after triggering on <code>measurement_done</code> . . . . . | 48 |
| 6.6 | Zybo Z7-10 board running the throughput measurement design. . . . .                                                                                                  | 49 |
| 6.7 | Vivado synthesis report: Dataflow Design <code>throughput_bench_top</code> . . . . .                                                                                 | 49 |
| 6.8 | Vivado implementation (post-place-and-route) utilization report. . . . .                                                                                             | 49 |
| 6.9 | Post-implementation timing summary showing worst negative slack at the<br>target clock frequency. . . . .                                                            | 50 |
| 7.1 | BER comparison across channel conditions: (a) AWGN, (b) GSM HT with-<br>out equalization, and (c) GSM HT with NLMS equalization. . . . .                             | 52 |
| C.1 | Vivado block design of the complete GMSK modem system. . . . .                                                                                                       | 60 |
| C.2 | Synthesized schematic of the <code>convEncoder_Seq</code> and <code>viterbi_decoder</code> mod-<br>ules. . . . .                                                     | 60 |

# List of Tables

|     |                                                                   |    |
|-----|-------------------------------------------------------------------|----|
| 3.1 | Consolidated system requirements. . . . .                         | 23 |
| 3.2 | Toolchain and target platform. . . . .                            | 24 |
| 3.3 | Verification matrix. . . . .                                      | 25 |
| 4.1 | Burst composition. . . . .                                        | 26 |
| 4.2 | Channel model parameters. . . . .                                 | 29 |
| 4.3 | Survivor management trade-off. . . . .                            | 30 |
| 6.1 | Defects found and fixed during RTL system simulation. . . . .     | 47 |
| 7.1 | Summary of communication performance results. . . . .             | 51 |
| 7.2 | HLS synthesis results for the hardware blocks. . . . .            | 52 |
| 7.3 | Measured system performance of the encoder–decoder chain. . . . . | 52 |
| A.1 | Principal MATLAB files by subsystem. . . . .                      | 57 |
| B.1 | HLS and RTL-support sources. . . . .                              | 59 |
| E.1 | Equalizer development summary (GSM HT channel). . . . .           | 62 |
| E.2 | RTL loopback measurement (100 MHz). . . . .                       | 62 |
| E.3 | On-board ILA counter readout (Zybo Z7-10, 50 MHz clock). . . . .  | 63 |

# Abbreviations

|      |                                          |
|------|------------------------------------------|
| ACS  | Add–Compare–Select                       |
| ADC  | Analogue-to-Digital Converter            |
| AWGN | Additive White Gaussian Noise            |
| AXI  | Advanced eXtensible Interface            |
| BER  | Bit Error Rate                           |
| BMU  | Branch Metric Unit                       |
| BRAM | Block Random Access Memory               |
| BSC  | Binary Symmetric Channel                 |
| BT   | Bandwidth–Time product                   |
| CPM  | Continuous Phase Modulation              |
| CSV  | Comma-Separated Values                   |
| DUT  | Device Under Test                        |
| FEC  | Forward Error Correction                 |
| FF   | Flip-Flop                                |
| FIFO | First-In First-Out buffer                |
| FIR  | Finite Impulse Response                  |
| FPGA | Field-Programmable Gate Array            |
| FSM  | Finite-State Machine                     |
| GMSK | Gaussian Minimum Shift Keying            |
| GSM  | Global System for Mobile Communications  |
| HLS  | High-Level Synthesis                     |
| HT   | Hilly Terrain (COST 207 channel profile) |
| II   | Initiation Interval                      |
| ILA  | Integrated Logic Analyzer                |
| IP   | Intellectual Property (core)             |
| ISI  | Intersymbol Interference                 |
| LFSR | Linear Feedback Shift Register           |
| LLR  | Log-Likelihood Ratio                     |
| LMS  | Least Mean Squares                       |
| LUT  | Look-Up Table                            |

|      |                                          |
|------|------------------------------------------|
| MLSE | Maximum Likelihood Sequence Estimation   |
| MMCM | Mixed-Mode Clock Manager                 |
| MSE  | Mean Squared Error                       |
| MSK  | Minimum Shift Keying                     |
| NLMS | Normalized Least Mean Squares            |
| NRZ  | Non-Return-to-Zero                       |
| PSD  | Power Spectral Density                   |
| RA   | Rural Area (COST 207 channel profile)    |
| RTL  | Register Transfer Level                  |
| SNR  | Signal-to-Noise Ratio                    |
| SPS  | Samples Per Symbol                       |
| TB   | Traceback                                |
| TBU  | Traceback Unit                           |
| TU   | Typical Urban (COST 207 channel profile) |
| VCD  | Value Change Dump                        |
| VIO  | Virtual Input/Output                     |
| XDC  | Xilinx Design Constraints                |

# Chapter 1

## Introduction

### 1.1 Background

Digital wireless communication systems must reconcile two conflicting demands: spectral efficiency, so that the transmitted signal occupies as little bandwidth as possible, and power efficiency, so that inexpensive, nonlinear power amplifiers can be used without distorting the signal. Constant-envelope modulation schemes address the second demand directly, because a signal whose amplitude never varies is immune to amplitude distortion in a saturated amplifier. Gaussian Minimum Shift Keying (GMSK) is the most commercially successful member of this family: it extends Minimum Shift Keying (MSK) with a Gaussian pre-modulation filter that smooths the frequency transitions between symbols, dramatically reducing out-of-band emissions at the cost of a controlled amount of intersymbol interference (ISI) [7]. GMSK was adopted as the physical-layer modulation of the GSM cellular standard, in which it operates at a symbol rate of 270.833 ksymbol/s with a bandwidth–time product of approximately 0.3 [17]. Reliable operation over noisy and fading radio channels additionally requires forward error correction. The rate-1/2 convolutional code with constraint length  $K = 7$  and generator polynomials  $(171, 133)_8$  is among the most widely deployed codes in existence, appearing in satellite standards, deep-space telemetry and terrestrial broadcast systems [11, 2]. Its maximum-likelihood decoder, the Viterbi algorithm [9, 10], is a canonical example of an algorithm whose regular trellis structure invites parallel hardware implementation, and whose hardware realization poses classical trade-offs between area, latency and throughput. Historically, such hardware has been written by hand at the register-transfer level (RTL). High-Level Synthesis (HLS) promises to raise the abstraction level, allowing the designer to express the algorithm in C++ and to control the generated micro-architecture through synthesis directives [25, 26]. Whether HLS can deliver correct, efficient implementations of communication blocks such as convolutional encoders and Viterbi decoders and what pitfalls arise in practice is a question best answered by carrying a complete design through the

full flow, from algorithm to measured hardware. That is the undertaking documented in this report.

## 1.2 Problem Statement

The project addresses the following engineering problem: *to design, model, evaluate and implement a complete GMSK burst modem, in which the channel-coding subsystem is realized in FPGA hardware through High-Level Synthesis, and to quantify its communication performance under both ideal and standardized mobile radio channel conditions as well as its hardware performance on a physical FPGA platform.* This problem decomposes into several coupled sub-problems, each of which required independent investigation:

1. construction of a bit-exact floating-point reference model of the entire transmit and receive chain in MATLAB;
2. development of a statistically sound BER evaluation methodology capable of resolving error rates down to the target of  $10^{-5}$ ;
3. extension of the channel model from AWGN to standardized GSM propagation environments, and the receiver enhancements (channel estimation, adaptive equalization, refined synchronization) that such channels necessitate;
4. translation of the encoder and decoder algorithms into synthesizable HLS C++ with industry-standard AXI4-Stream interfaces;
5. rigorous verification of the generated hardware at the C, RTL co-simulation and on-board levels; and
6. measurement and critical analysis of the throughput, latency and resource utilization of the resulting hardware.

## 1.3 Motivation

Three motivations drove the selection of this project. First, GMSK sits at an instructive intersection of theory and practice: it is analytically rich (continuous phase, modulation memory, ISI introduced deliberately by the pulse-shaping filter) yet grounded in a real, massively deployed standard, which allows the work to be validated against published specifications such as 3GPP TS 45.004 and TS 45.005 [17, 18]. Second, the Viterbi decoder is a demanding and well-understood hardware target, making it an ideal vehicle for evaluating the practical maturity of the HLS design flow: the quality of the result can be judged objectively through initiation interval, resource utilization and measured

throughput. Third, the progression from an idealized simulation to a receiver that survives a severe standardized fading channel exercises the full breadth of receiver engineering, including synchronization, estimation and equalization. It also exposes the numerous subtle alignment and reference-signal problems that only appear when these subsystems interact.

## 1.4 Research Objectives

The objectives established at the outset of the project were as follows:

1. To study the theory of continuous phase modulation, GMSK, convolutional coding and Viterbi decoding to the depth required for independent implementation.
2. To implement a complete GMSK packet modem in MATLAB, including framing, coding, pulse shaping, modulation, demodulation, synchronization and soft-decision decoding, and to verify end-to-end data recovery under AWGN.
3. To evaluate the modem's BER as a function of SNR with statistically defensible methodology, and to determine the SNR required for a BER of  $10^{-5}$ .
4. To identify a real product and standardized channel model appropriate to GMSK, and to evaluate the modem under that channel, extending the receiver with channel estimation and adaptive equalization as required.
5. To implement the convolutional encoder and a 64-state Viterbi decoder in Vitis HLS with 32-bit AXI4-Stream interfaces, meeting the interface requirement imposed on the project.
6. To verify the hardware implementations against golden references through C simulation and RTL co-simulation.
7. To measure the throughput of the encoder-decoder chain both in RTL simulation and on the Zybo Z7-10 FPGA board, and to identify the performance bottleneck.
8. To document all design alternatives, trade-offs, failures and lessons learned so that the engineering process itself is assessable.

## 1.5 Project Scope

The scope encompasses the complete MATLAB modem, its BER evaluation under three channel conditions (AWGN, GSM hilly terrain without equalization, GSM hilly terrain with NLMS equalization), and the HLS implementation, verification and hardware

throughput measurement of the convolutional encoder and Viterbi decoder. A synthesizable HLS implementation of the GMSK phase mapper was additionally developed and verified in C simulation. Full hardware realization of the remaining modem blocks (Gaussian filter, demodulator, synchronizer, equalizer) was investigated at the architectural level but is outside the implemented scope and is identified as future work. Radio-frequency hardware, antennas and over-the-air testing are excluded; the channel is modelled digitally throughout.

## 1.6 Expected Contributions

The work contributes: (i) a complete, staged reference implementation of a GMSK burst modem whose receiver demonstrably operates over the severe COST 207 hilly-terrain channel; (ii) a documented, reproducible methodology for BER evaluation with explicit stopping rules and target-crossing interpolation; (iii) a practical case study of HLS-based implementation of channel-coding hardware, including a catalogue of HLS-specific pitfalls (top-level pipelining, unrolling of state-dependent loops, automatic AXI register slices) discovered through systematic co-simulation debugging; (iv) a self-contained on-chip throughput measurement architecture with LFSR stimulus and ILA instrumentation; and (v) quantified hardware results 32.65 Mb/s measured decoded throughput, 84-cycle pipeline latency, and per-block resource figures with a critical analysis identifying the Viterbi traceback decoder as the bottleneck.

## 1.7 Report Structure and Chapter Summary

Chapter 2 reviews the theoretical and technological literature underpinning the project. Chapter 3 states the system requirements and describes the development methodology and toolchain. Chapter 4 presents the system design of every transmitter, receiver and hardware subsystem, including the architectural alternatives that were considered and rejected. Chapter 5 details the implementation of the MATLAB model and the HLS hardware, including all optimization directives and their rationale. Chapter 6 describes the verification and experimental programme, from unit-level golden-model testing to on-board measurement, and documents the major debugging investigations. Chapter 7 presents and critically discusses the results. Chapter 8 concludes and outlines future work. The appendices catalogue source files, simulation setups and experimental data.

# Chapter 2

## Literature Review

### 2.1 Digital Communication Systems

The architecture of a digital communication system (source, channel encoder, modulator, channel, demodulator, channel decoder, sink) was formalized in the decades following Shannon’s founding work [1] and is treated comprehensively in the standard texts of Proakis and Salehi [2], Sklar [3] and Haykin [4]. Three orthogonal performance metrics recur throughout this project and must be distinguished carefully: the bit error rate (BER), which measures reliability; latency, which measures the delay from input to output; and throughput, which measures the rate of information transfer. A design may exhibit high latency yet high steady-state throughput once its pipeline is full; conflating these metrics leads to incorrect conclusions, a point that became directly relevant during the hardware evaluation of Chapter 6.

### 2.2 Continuous Phase Modulation and Minimum Shift Keying

Continuous Phase Modulation (CPM) is a family of constant-envelope schemes in which the information is carried in the phase trajectory of the carrier, and the phase is constrained to be continuous across symbol boundaries [5]. Continuity of phase eliminates the spectral splatter associated with abrupt phase transitions and yields signals whose envelope is strictly constant, making CPM signals robust to nonlinear amplification. Minimum Shift Keying (MSK) is the simplest binary CPM, equivalent to continuous-phase frequency shift keying with modulation index  $h = 0.5$ , the minimum index for which the two signalling frequencies remain orthogonal [6]. MSK achieves the same asymptotic error performance as antipodal signalling while possessing a constant envelope; however, its rectangular frequency pulse still produces relatively slowly decaying spectral sidelobes.

## 2.3 Gaussian Minimum Shift Keying

GMSK, introduced by Murota and Hirade [7], prefilters the non-return-to-zero data stream with a Gaussian low-pass filter before frequency modulation. The filter is characterized by its bandwidth–time product  $BT$ : smaller values of  $BT$  compact the spectrum further but spread each symbol’s frequency pulse over more symbol intervals, deliberately introducing intersymbol interference and hence *modulation memory*. GSM specifies  $BT \approx 0.3$  [17]; the present project adopts  $BT = 0.35$ , a nearby operating point with slightly less ISI. Because of this memory, every transmitted sample depends on several neighbouring bits, a property with far-reaching consequences for receiver design, most notably for the generation of equalizer training references, as discovered experimentally in this project (Chapter 6). Demodulation of GSMK may be coherent or non-coherent. The non-coherent phase-derivative (limiter–discriminator style) approach, in which the receiver computes the instantaneous phase of the complex baseband signal, unwraps it and differentiates it to recover the frequency deviation, offers implementation simplicity and robustness to phase reference errors, and is the approach adopted here. Because GSMK’s controlled ISI makes the signal a form of partial-response signalling, sequence detection techniques such as the Viterbi algorithm are naturally suited to GSMK reception [5, 2].

## 2.4 Convolutional Coding

Convolutional codes, introduced by Elias [8], encode a continuous stream of information bits through a shift register and modulo-2 adders defined by generator polynomials. A rate-1/2 code with constraint length  $K$  produces two coded bits per information bit and possesses  $2^{K-1}$  internal states; the code used throughout this project,  $K = 7$  with generators  $(171, 133)_8$ , has 64 states and a free distance of 10, and is the de facto industry-standard code adopted by CCSDS, DVB and numerous satellite systems [11, 2]. Crucially for hardware architecture, the encoder is a finite-state machine: its output depends on the current input *and* the accumulated state. This sequential dependency fundamentally limits naive parallelization of the encoder, an issue examined at length during the hardware architecture exploration of Chapter 4. Look-ahead transformations that re-derive Boolean equations to process multiple bits per cycle exist [23] but at significant combinational cost.

## 2.5 The Viterbi Algorithm

Viterbi’s algorithm [9] performs maximum-likelihood sequence estimation over the code trellis with complexity linear in the sequence length, and Forney [10] established both its optimality and the trellis formalism used in every implementation since. Each trellis stage involves, for every state, an Add–Compare–Select (ACS) operation over the two in-

coming branches; the winning predecessor is recorded as a one-bit *survivor* decision. Two survivor-management architectures dominate the hardware literature: *register exchange*, in which every state carries its full decoded path (minimal latency, prohibitive register cost for large state counts), and *traceback*, in which only survivor bits are stored and the decoded sequence is recovered by walking backwards through the survivor memory (industry standard for  $K = 7$  decoders) [21, 22]. The traceback depth required for negligible truncation loss is commonly taken as approximately five times the constraint length [11], i.e. about 35 for  $K = 7$ . Sliding-window organizations of the survivor memory into rotating banks permit continuous streaming decoding with bounded memory [22].

## 2.6 Soft-Decision Decoding

Hard-decision decoding discards the confidence of each received symbol; soft-decision decoding retains it, conventionally as log-likelihood ratios (LLRs), and yields a coding-gain improvement of approximately 2 dB for convolutional codes on the AWGN channel [2, 12]. For binary antipodal signalling over AWGN, the maximum-likelihood branch metric reduces to a correlation between the expected symbol polarity and the received LLR, which is the metric adopted in the MATLAB reference decoder of this project. Quantization of soft inputs to 3–4 bits sacrifices only fractions of a decibel [13], which justifies the project’s methodological choice of keeping the MATLAB reference in floating point (as the ideal benchmark) and deferring quantization to the hardware implementation.

## 2.7 Mobile Radio Channels

The mobile radio channel is characterized by multipath propagation, giving rise to frequency-selective fading, Doppler spread due to motion, and shadowing [14, 15]. In non-line-of-sight conditions the envelope of a narrowband received signal is Rayleigh-distributed; deep fades cause bursts of errors that are qualitatively different from the independent errors of AWGN and that degrade Viterbi decoding unless mitigated through interleaving, diversity or equalization [2, 14]. For GSM, the COST 207 action defined standardized tapped-delay-line channel profiles such as Typical Urban (TU), Rural Area (RA), Hilly Terrain (HT) and Bad Urban (BU) each specifying path delays, powers and Doppler spectra [16]; conformance testing against these profiles is embodied in 3GPP TS 45.005 [18]. Training-sequence-based channel estimation and adaptive equalization, typically least-squares estimation and LMS/NLMS or decision-feedback structures [19, 20], are the standard receiver countermeasures and are the ones developed in this project.

## 2.8 FPGA Implementation of Communication Systems

FPGAs are the platform of choice for physical-layer prototyping because their fine-grained parallelism matches the structure of signal-processing datapaths [24]. Viterbi decoders in particular have a long FPGA implementation history, with the parallel ACS array, survivor memory organization and traceback scheduling as the principal design axes [21, 22]. Streaming dataflow between blocks is conventionally carried over the AXI4-Stream protocol, in which a transfer occurs exactly when `TVALID` and `TREADY` are simultaneously asserted, with `TLAST` delimiting packets [27]. Correct implementation of this handshake in particular, never gating `TVALID` on `TREADY` is a recurring source of deadlocks and data loss, as this project experienced directly (Chapter 6).

## 2.9 High-Level Synthesis

High-Level Synthesis compiles C/C++ descriptions into RTL, with the micro-architecture steered by directives (pragmas) controlling pipelining, loop unrolling, array partitioning and interface synthesis [25, 26]. The key quality metric of a pipelined loop is its initiation interval (II): the number of cycles between successive loop iterations, with  $II = 1$  denoting acceptance of new data every cycle. The literature and vendor documentation emphasize that HLS does not remove the need for hardware thinking: loop-carried dependencies limit achievable II, unrolling stateful loops creates long combinational chains, and interface registering interacts subtly with block-level handshakes [26]. The present project provides empirical confirmation of each of these effects, documented as a catalogue of HLS pitfalls in Chapters 5 and 6.

## 2.10 Existing Implementations

Commercial and open IP cores exist for both convolutional encoding and Viterbi decoding (e.g. the Xilinx Viterbi Decoder LogiCORE [28]), and FPGA realizations of GMSK and related CPM modems are well established in the applied literature and in vendor application material [24]. These implementations, however, are typically delivered as opaque RTL. Educational and research value lies in traversing the full path algorithm, reference model, HLS description, co-simulation, hardware measurement with every design decision exposed, which is precisely the gap this project addresses.

## 2.11 Research Gaps Addressed

Three gaps identified during the review shaped the project. First, while HLS tutorials abound, few end-to-end case studies document the *failure modes* of the HLS flow for communication blocks with state (encoders) and complex control (traceback decoders); this project documents them systematically. Second, receiver-side literature usually treats synchronization, channel estimation and equalization in isolation; the interaction defects that arise when they are composed (group-delay accounting across index domains, reference-signal contamination by modulation memory) are rarely reported, yet consumed a substantial fraction of this project’s engineering effort and are analysed here in detail. Third, published BER results frequently omit the statistical methodology (stopping rules, confidence in the low-BER tail); this project makes the methodology explicit.

## 2.12 Chapter Summary

The literature establishes GMSK as a constant-envelope partial-response modulation with memory, the  $(171, 133)_8$  code and traceback Viterbi decoding as industry standards, COST 207 profiles as the correct evaluation channels for a GSM-class modem, and HLS as a productive but pitfall-laden route to hardware. The remainder of the report documents how these elements were independently synthesized into a working system.

# Chapter 3

## System Requirements and Design Methodology

### 3.1 Overall Modem Architecture

The target system is a burst-mode GMSK modem whose end-to-end signal flow is:

Information bits → Convolutional encoder → Packet builder → Gaussian filter  
→ GMSK mapper → Channel → Coarse demodulation and preamble detection  
→ Burst extraction → (Channel estimation / equalization) → Demodulation  
→ Payload extraction → Soft-decision Viterbi decoder → Recovered bits

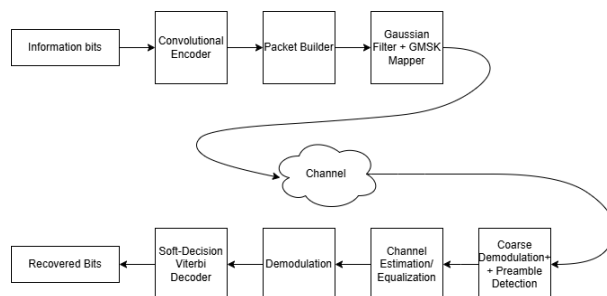


Figure 3.1: Overall GMSK modem architecture and end-to-end signal flow.

### 3.2 Requirements

The requirements, consolidated from the project brief and supervisor direction, are listed in Table 3.1. Requirement R7 deserves comment because of its architectural consequences. A 32-bit streaming interface does not imply 32-bit-wide internal processing; the separation of interface width from algorithmic processing width became one of the central architectural insights of the project (Section 4.10).

Table 3.1: Consolidated system requirements.

| ID  | Requirement                                                                 | Origin        |
|-----|-----------------------------------------------------------------------------|---------------|
| R1  | Rate-1/2 convolutional coding, $K = 7$ , generators (171, 133) <sub>8</sub> | Design choice |
| R2  | GMSK modulation, $BT = 0.35$ , 4 samples per symbol                         | Design choice |
| R3  | Burst framing with preamble and training fields                             | Design choice |
| R4  | Soft-decision Viterbi decoding at the receiver                              | Design choice |
| R5  | BER evaluated versus SNR; target operating point $BER = 10^{-5}$            | Project brief |
| R6  | Evaluation under a standardized channel model of a real GMSK product        | Supervisor    |
| R7  | Hardware blocks expose 32-bit AXI4-Stream input and output interfaces       | Supervisor    |
| R8  | Equalizer training uses only known training bits (no payload knowledge)     | Supervisor    |
| R9  | Encoder and decoder implemented via Vitis HLS and verified in co-simulation | Project brief |
| R10 | Throughput of the hardware chain measured on the Zybo Z7-10 board           | Project brief |

### 3.3 Development Methodology

The project followed a staged, model-based methodology in which each stage produced a verified artefact that served as the golden reference for the next:

1. **Floating-point reference modelling.** Every algorithm was first implemented and verified in MATLAB. The MATLAB implementation is deliberately ideal floating-point arithmetic, no quantization, so that it bounds the achievable performance and provides bit-exact stimuli and expected responses for hardware verification.
2. **Staged channel realism.** The channel model was upgraded incrementally (AWGN  $\rightarrow$  flat Rayleigh  $\rightarrow$  standardized GSM multipath), the receiver being extended at each stage only as far as the new impairments demanded. The validation sequence AWGN  $\rightarrow$  static multipath  $\rightarrow$  slow fading  $\rightarrow$  fast fading  $\rightarrow$  estimation  $\rightarrow$  equalization was adopted explicitly after an early experiment that skipped intermediate stages produced an uninterpretable total failure (Section 6.5).
3. **HLS implementation against golden references.** Each hardware block was translated from its MATLAB reference into synthesizable C++, verified first in C simulation against a pure-software golden model, then in RTL co-simulation, and finally in hardware.
4. **Measurement-driven evaluation.** Performance claims (BER, throughput, la-

tency, II) were established by instrumented measurement Monte Carlo simulation with stopping rules for BER; cycle-accurate counters in RTL simulation and on-chip ILA capture for hardware metrics rather than by estimation alone.

### 3.4 Toolchain

Table 3.2: Toolchain and target platform.

| Tool / platform                           | Role                                                                                                       |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------|
| MATLAB + Communications Toolbox           | Reference modelling; channel models ( <code>stdchan</code> , <code>physconst</code> ); BER experimentation |
| Xilinx Vitis HLS 2025.2                   | C++-to-RTL synthesis of encoder, decoder and GMSK mapper; C simulation and RTL co-simulation               |
| Xilinx Vivado 2025.2                      | RTL simulation (XSIM), synthesis, implementation, ILA insertion, bitstream generation, Hardware Manager    |
| Digilent Zybo Z7-10                       | Target board: Zynq-7000 FPGA, 125 MHz oscillator on pin K17                                                |
| Git                                       | Version control of all sources and testbenches                                                             |
| Visual Studio Code                        | HLS C++ development, integrated with the Vitis environment                                                 |
| L <sup>A</sup> T <sub>E</sub> X (latexmk) | Project reporting                                                                                          |
| Python                                    | LUT generation for the GMSK mapper; structural checks of generated testbenches                             |

A practical environment integration step is worth recording: Vitis HLS development was carried out in VS Code rather than the Vitis IDE by running the vendor environment script (`settings64.bat`) and launching `code .` from the same configured terminal, with `c_cpp_properties.json` pointing IntelliSense at the Vitis and HLS include directories. This provided code completion and header navigation for `ap_int` and `hls::stream` types. A benign but initially alarming compiler warning (`__GMP_LIBGMP_DLL` macro redefinition) was traced to conflicting definitions inside Xilinx’s own headers and shown to be harmless to synthesis.

### 3.5 Verification Flow

Verification was layered, with each layer catching a different defect class, as summarized in Table 3.3.

Table 3.3: Verification matrix.

| Level                 | Method                                                                    | Defects caught                                                     |
|-----------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------|
| Algorithm             | Standalone C++/MATLAB golden-model comparison                             | Algorithmic errors; block-boundary state assumptions               |
| HLS C simulation      | Self-checking testbenches (41 encoder tests; 10,000-bit decoder BER test) | Functional bugs; interface packing errors                          |
| RTL co-simulation     | Vitis HLS co-sim (XSIM)                                                   | Pragma-induced handshake defects; register-slice latency           |
| RTL system simulation | SystemVerilog loopback testbenches with counters and VCD                  | AXI protocol violations; multiple-driver races; measurement errors |
| Hardware              | ILA-instrumented measurement on Zybo Z7-10                                | Constraint and debug-infrastructure issues; final performance      |

## 3.6 Engineering Workflow

Day-to-day work alternated between investigation (literature and architecture study), implementation, and experiment, with all significant artefacts committed to version control (representative commits: 17baf07, ec99443 for throughput testbenches). AI code assistants were employed as productivity tools for generating boilerplate testbenches and first-draft conversions; a deliberate practice was developed of writing complete engineering specifications (interfaces, protocols, measurement definitions, synthesizability constraints) rather than terse requests, and of auditing all generated code, an audit that repeatedly discovered substantive defects (bit-serial driving of word interfaces, incorrect output-bit counting, simulation race conditions), confirming that responsibility for correctness remained with the engineer.

### 3.6.1 Organisational Context

The project was carried out at Thakshana Technologies (PVT) Limited as part of the industrial training programme of the University of Moratuwa. Technical direction was provided by the Senior DSP Engineer as direct supervisor, with overall supervision by the company Director and milestone coordination by the Project Manager.

# Chapter 4

## System Design

This chapter presents the design of every subsystem, together with the alternatives considered and the rationale for each decision. The design is described in its final form, but design iterations are indicated where the final form emerged from an earlier, rejected approach; the experimental route to those rejections is given in Chapter 6.

### 4.1 Frame Structure

The modem is burst-oriented. A single burst, defined by the `Waveform` class (waveform identifier 0), comprises 624 bits: The payload and training fields are interleaved after the

Table 4.1: Burst composition.

| Field             | Length   | Purpose                                                 |
|-------------------|----------|---------------------------------------------------------|
| Preamble          | 48 bits  | Burst detection and synchronization; equalizer training |
| Encoded payload   | 544 bits | 272 information bits (34 bytes) after rate-1/2 coding   |
| Training sequence | 32 bits  | Four interleaved 8-bit chunks for channel estimation    |
| Total             | 624 bits | = 2496 complex samples at 4 samples/symbol              |

preamble in the pattern  $[\text{Preamble} \mid P_1 \mid T_1 \mid P_2 \mid T_2 \mid P_3 \mid T_3 \mid P_4 \mid T_4]$ , distributing the training bits across the burst so that a time-varying channel can in principle be tracked. A `bitMapMask` vector labels every bit position with its field type (1 = preamble, 2 = payload, 3 = training); the receiver uses this mask both for payload extraction and for deriving training-sample indices. A subtle but consequential defect was discovered in the waveform definition: the preamble and training sequences were generated by two successive calls to a random-sequence generator that each reset the random seed (`rng(12345)`), with the result that the first 32 bits of the preamble and the training sequence were *identical*. This silently degraded both preamble correlation and equalizer training. The corrected design seeds the generator once, draws all bits in a single call, and partitions them. This demonstrates

how a seemingly trivial two-line initialization error can introduce unintended correlations between otherwise independent data streams, leading to misleading simulation results.

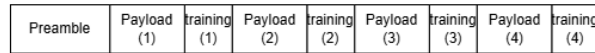


Figure 4.1: Burst structure: preamble, interleaved payload and training chunks

## 4.2 Convolutional Encoder

The encoder implements the rate-1/2,  $K = 7$  code with generators  $G_1 = (171)_8$  and  $G_2 = (133)_8$ . In binary,  $G_1 = 1111001$  (taps at positions  $\{1, 2, 3, 4, 7\}$ ) and  $G_2 = 1011011$  (taps at  $\{1, 3, 4, 6, 7\}$ ). The encoder maintains a 6-bit state register; for each input bit  $b$  the two coded bits are

$$g_1 = b \oplus s_0 \oplus s_1 \oplus s_2 \oplus s_5, \quad (4.1)$$

$$g_2 = b \oplus s_1 \oplus s_2 \oplus s_4 \oplus s_5, \quad (4.2)$$

after which the state updates as  $s \leftarrow ((s \ll 1) \& 0\mathbf{x}3\mathbf{F}) \mid b$ . The output stream interleaves the two generator outputs as  $[g_1^{(0)}, g_2^{(0)}, g_1^{(1)}, g_2^{(1)}, \dots]$ . Because the encoder is a state machine, encoded blocks are *not* independent unless the state is explicitly managed at block boundaries, the root cause of a major verification incident analysed in Section 6.10.

## 4.3 Gaussian Filter

The pulse-shaping filter is a 33-tap FIR with  $BT = 0.35$ , span 8 symbols and 4 samples per symbol, generated from a direct Gaussian expression (rather than the textbook Q-function frequency pulse) and applied by full convolution, producing Q15 fixed-point output of length  $N_{\text{bits}} \times \text{SPS} + (N_{\text{taps}} - 1)$ . Its group delay,

$$g_d = \left\lfloor \frac{\text{span} \times \text{SPS}}{2} \right\rfloor = 16 \text{ samples},$$

propagates into every index calculation in the receiver. Three independent receiver defects were eventually traced to inconsistent group-delay accounting (Section 6.8), making this single constant arguably the most bug-prone quantity in the system. The specific (non-standard) tap formula also has a design consequence: any receiver-side reference regeneration must reuse the *same* filter object, because even small pulse-shape mismatches constitute a systematic error that adaptive equalization cannot remove.

## 4.4 GMSK Modulator (Mapper)

The mapper converts the filtered sequence into a unit-envelope complex baseband signal by phase accumulation and table lookup. Its fixed-point datapath is: (i) right-shift of each filtered sample by 8 bits (gain reduction by 256); (ii) accumulation with asymmetric saturation (threshold  $\pm 65535$ , clamped to  $\pm 32768$ ); (iii) reduction of the accumulator modulo the table length to index a 512-entry Q15 sine/cosine lookup table (positive values clamped to 0.999969482421875 before scaling by 32767); (iv) emission of  $I + jQ$  with amplitude  $\approx 32767$ . The phase accumulator is reset at the start of each burst. Integer phase accumulation with LUT synthesis is the standard hardware-friendly realization of continuous phase modulation, and the identical structure was later carried into HLS (Section 5.5), where the semantic difference between MATLAB's always-positive `mod` and C++'s truncating `%` required explicit correction.

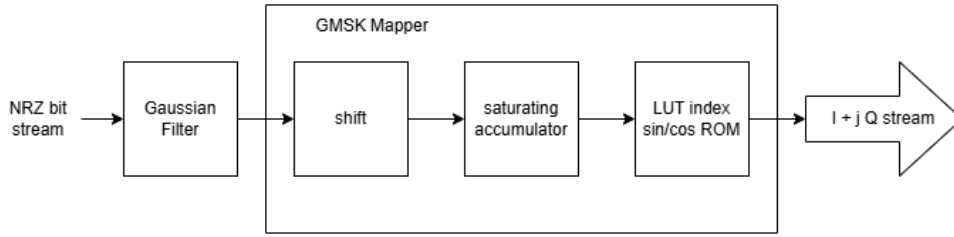


Figure 4.2: GMSK mapper datapath: shift, saturating accumulator, LUT index, sin/cos ROM

## 4.5 Channel Models

Three channel models of increasing realism were designed into the simulation environment.

### 4.5.1 AWGN

$r(t) = s(t) + n(t)$ , with noise power set directly from the SNR definition,  $P_n = P_s / 10^{\text{SNR}_{\text{dB}}/10}$ . This is the algorithm-verification channel: no distortion, constant gain, perfect effective synchronization.

### 4.5.2 Flat Rayleigh fading

$r = hs + n$  with  $h = (\mathcal{N}(0, 1) + j\mathcal{N}(0, 1))/\sqrt{2}$ , giving a Rayleigh-distributed envelope and uniform phase. One coefficient per burst (block fading) was adopted initially. The design intent of this stage was to introduce, in isolation, the three phenomena absent from AWGN: amplitude scaling, phase rotation, and deep fades producing burst errors.

### 4.5.3 Standardized GSM multipath

The principal evaluation channel is the COST 207 GSM Hilly Terrain profile with 12 taps, instantiated through the Communications Toolbox:

```
v = 120*1e3/3600;           % mobile speed: 120 km/h
fc = 1.8e9;                 % GSM-1800 carrier
fd = v*fc/physconst('lightspeed'); % max Doppler ~ 200 Hz
Rsym = 270.833e3;           % GSM symbol rate
Rsamp = SPS * Rsym;         % ~1.083 MHz at SPS = 4
gsmChan = stdchan('gsmHTx12c1', Rsamp, fd);
```

The channel applies a tapped-delay-line Rayleigh process with the standardized delays, powers and classical Doppler spectrum, i.e.  $r(n) = h(n)*x(n)+w(n)$  where the convolution introduces genuine frequency selectivity and ISI. Operational design rules established during integration: the channel object must be **reset** between Monte Carlo trials (it carries internal fading state); channel output power should be normalized so the SNR definition remains meaningful; and initial bring-up should use reduced Doppler and milder profiles (TU) before the severe HT case. Table 4.2 summarizes the channel parameters.

Table 4.2: Channel model parameters.

| Parameter         | AWGN       | Flat Rayleigh     | GSM HT ( <b>gsmHTx12c1</b> )          |
|-------------------|------------|-------------------|---------------------------------------|
| Multipath taps    | —          | 1                 | 12 (COST 207 HT)                      |
| Fading statistics | —          | Rayleigh, block   | Rayleigh, time-varying                |
| Doppler           | —          | 0 (block)         | $\approx 200$ Hz (120 km/h @ 1.8 GHz) |
| Sample rate       | $4R_{sym}$ | $4R_{sym}$        | 1.0833 MHz                            |
| Noise             | AWGN       | AWGN after fading | AWGN after fading                     |

## 4.6 Synchronization

Burst synchronization is achieved by bipolar cross-correlation of the demodulated soft-metric (LLR) stream against the known 48-bit preamble, followed by non-maximum suppression with a configurable threshold and minimum inter-detection spacing. The design must respect two index-domain subtleties. First, the LLR stream begins only after the demodulator has discarded the 16-sample group delay, so LLR index 1 corresponds to received sample 17; the burst start in the sample domain is therefore

$$n_{\text{samp}} = g_d + (n_{\text{sym}} - 1) \text{SPS} + 1.$$

Second, the correlation peak position itself includes the deterministic filter delay: a detected preamble index of 4 (sample 13) was initially suspected as an error but shown

to be the natural consequence of Gaussian filtering and symbol-rate sampling. A two-stage (coarse/fine) structure resolves the circular dependency between equalization and synchronization: a coarse demodulation pass provides timing only; the burst is then re-extracted from the *complex* sample stream for equalization and a second, clean demodulation. Missed detections are charged as fully erroneous payloads in BER accounting; the design anticipates (as future work) adaptive or normalized correlation thresholds for low-SNR robustness.

## 4.7 Demodulator

The demodulator is non-coherent: it computes the instantaneous phase of the complex baseband via `atan2`, unwraps it, differentiates it to obtain the per-sample frequency deviation, compensates the filter group delay by discarding 16 samples at each end, and samples at symbol instants to produce one real soft metric per burst bit. These metrics serve directly as LLRs for the soft-decision decoder. A cosmetic but instructive detail: a scale factor of  $1/2$  applied before `atan2`, documented in the legacy code as a Q15-to-Q14 conversion, is mathematically irrelevant (a common scaling of  $I$  and  $Q$  does not alter their angle) and was flagged for removal.

## 4.8 Viterbi Decoder Architecture

### 4.8.1 Architecture selection

Two survivor-management architectures were evaluated. Register exchange offers minimal latency but, for 64 states and a 32-stage history, requires on the order of  $64 \times 32$  path registers with corresponding routing and power cost. Traceback stores a single survivor bit per state per stage and recovers the path retrospectively, at the cost of decode latency. Traceback was selected as the industry-standard choice for  $K = 7$ ; the decision is summarized in Table 4.3.

Table 4.3: Survivor management trade-off.

| Criterion              | Register exchange                      | Traceback (selected)    |
|------------------------|----------------------------------------|-------------------------|
| Survivor storage       | $2^{K-1} \times$ path length registers | 1 bit/state/stage (RAM) |
| Latency                | Minimal                                | Traceback depth         |
| Power                  | High (mass register copying)           | Lower                   |
| Scalability to $K = 7$ | Poor                                   | Good; industry standard |

## 4.8.2 Functional blocks

The decoder pipeline is:

symbols  $\rightarrow$  BMU  $\rightarrow$  ACS $\times$ 64  $\rightarrow$  survivor memory  $\rightarrow$  minimum-metric search  $\rightarrow$  traceback  $\rightarrow$  bits.

**Branch Metric Unit.** For each of the four possible code symbols the BMU computes the distance to the received symbol: Hamming distance for the hard-decision hardware decoder; the correlation metric  $-\sum \hat{c}_i \Lambda_i$  (expected polarity  $\hat{c}_i \in \{+1, -1\}$  against received LLR  $\Lambda_i$ ) for the soft-decision reference. In the studied legacy RTL, the BMU computes the four combinations  $\pm g_0 \pm g_1$  of two soft samples; an important lesson from reverse-engineering that code is that hardware branch-metric labels are optimized for the radix-2 butterfly wiring and are not a direct one-to-one labelling of encoder codewords. **ACS.** Every next state  $ns$  has exactly two predecessors,  $p_0 = (ns_{[4:0]} \ll 1)$  and  $p_1 = p_0 + 1$ ; the input bit associated with the transition is  $ns_{[5]}$ . The ACS adds path and branch metrics for both candidates, compares, selects the survivor, and emits one survivor bit. Path metrics are initialized with state 0 at zero and all others at the maximum (or most-negative, in max-metric conventions), encoding the knowledge that the encoder starts from the all-zero state. Metric growth is contained either by saturating arithmetic (hardware decoder) or subtract-minimum normalization (parameterized variant). **Survivor memory and sliding window.** Survivor bits are organized as `survivor[depth][states]`. For continuous streaming, the studied RTL organizes the memory as four rotating 16-symbol banks; any two adjacent full banks form a 32-symbol traceback window (A+B, B+C, C+D, D+A), bounding memory while allowing decoding to proceed concurrently with reception. The banks are a hardware-organization device, not part of the algorithm. **Traceback.** Starting from the state with the best terminal metric, the unit iteratively reads the survivor bit, reconstructs the predecessor as  $(s_{[4:0]} \ll 1) \mid \text{survivor}$ , and emits the decoded bit  $s_{[5]}$ , walking backwards through the window. The traceback depth follows the  $\approx 5K$  rule (35 for  $K = 7$ ); 32 was adopted as a binary-friendly compromise.

## 4.8.3 Block-mode versus sliding-window operation

The hardware decoder operates in fixed blocks: one 32-bit AXI word carries 16 two-bit symbols and yields 16 decoded bits. The MATLAB reference operates as a continuous sliding-window decoder over arbitrary frame lengths, emitting one bit per stage once the window fills. A sliding decoder without termination produces only  $N - T_B + 1$  bits from  $N$  stages; three end-of-frame strategies were analysed: zero-tail termination (append  $K - 1$  zeros, forcing the encoder to state 0; standard in DVB/CCSDS; recommended), final traceback flush with progressively shorter depths (full length, degraded tail reliability), and enlarged traceback depth (cannot remove the boundary effect) with zero-tail

termination selected as the target design.

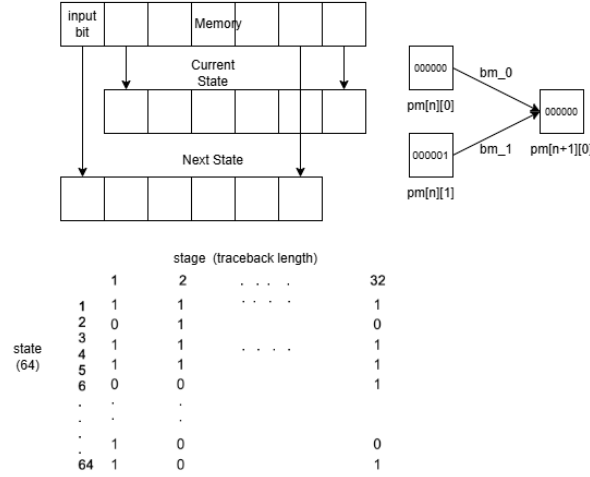


Figure 4.3: Viterbi decoder architecture: BMU, 64-way ACS array, survivor memory banks, minimum-metric search, traceback unit

## 4.9 Receiver Equalization Design

The equalization subsystem underwent the most extensive design iteration of the project; the full experimental narrative is in Section 6.6, and only the final design rationale is given here. **Placement.** The equalizer operates on complex baseband samples *before* demodulation (“Option B”). The alternative equalizing the real-valued post-demodulation metric stream (“Option A”) was rejected because the multipath channel acts on the complex signal, and LLRs, being decision statistics, cannot be meaningfully equalized. **Algorithm.** A complex normalized LMS (NLMS) transversal equalizer with weight update

$$\mathbf{w} \leftarrow \mathbf{w} + \frac{\mu}{\|\mathbf{u}\|^2 + \varepsilon} \mathbf{u} e^*, \quad e = d - \mathbf{w}^H \mathbf{u},$$

adapting only at known reference positions (burst-mode training-directed adaptation) and filtering throughout. A decision delay  $D$  aligns the reference with delayed multipath energy: training against  $d(i) = s_{\text{ref}}(i - D)$  with  $D$  near the filter centre. **Reference signal.** Three candidate references were considered. Using the transmitted waveform itself is physically unrealizable (it embeds the unknown payload). Using the interleaved mid-burst training chunks; the initially mandated approach is corrupted by GMSK modulation memory: the complex samples over the training positions depend on adjacent unknown payload bits. The final design trains on the *contiguous preamble*, whose waveform is known exactly and independent of payload, regenerated bit-exactly by passing the known bits through the very same **GaussianFilter** and **GmskMapper** objects used in the transmitter (a float reimplementaion was rejected: the non-standard Gaussian tap formula and the 512-point integer LUT would each inject systematic reference error that

adaptation cannot cancel). The reference is trimmed by the filter transient ( $g_d = 16$  samples) and normalized to unit amplitude by its median magnitude. **Complementary single-tap path.** For the flat-Rayleigh stage, a least-squares single-tap estimator over training positions,

$$\hat{h} = \frac{\sum_i r_i s_i^*}{\sum_i |s_i|^2}, \quad r_{eq} = r/\hat{h},$$

was designed as a separate modular path (channel estimate, one-tap equalizer, dedicated BER driver), sufficient for frequency-flat fading and pedagogically prior to the adaptive equalizer.

## 4.10 Hardware Architecture and AXI4-Stream Interfaces

### 4.10.1 Interface requirement and the parallelization study

Requirement R7 fixes 32-bit AXI4-Stream interfaces. Since the code is rate-1/2, one 32-bit input word must yield two 32-bit output words. An architecture study was conducted on how to organize the encoder internally. The first candidate, proposed and developed in detail before being rejected, split each input word into two 16-bit halves processed by two parallel encoder cores, with the second core's 6-bit state initialized from the last six bits of the first half (correct, because the encoder state *is* its shift register), and with output buffering to serialize the two half-results onto the single output stream. Critical analysis dismantled the proposal: the encoder is a sequential FSM, so the second core cannot start until the first core's final state is known, so the true speed-up is far below  $2\times$ ; and the design doubles the core plus adds state-transfer, buffering and scheduling logic, a poor area/complexity bargain. A look-ahead reformulation (re-derived Boolean equations processing multiple bits per cycle) was identified as the only genuine algorithmic parallelization, and set aside on complexity grounds. The decisive insight was the *separation of interface width from processing width*: commercial IP routinely presents wide stream interfaces around internally serial or deeply pipelined datapaths. The final architecture registers the 32-bit input word, iterates a single bit-serial encoder core 32 times (2 coded bits per iteration), assembles the 64 coded bits in a buffer, and emits two 32-bit words, meeting R7 exactly while keeping the datapath minimal. Throughput is recovered through temporal parallelism (loop pipelining) rather than spatial duplication.

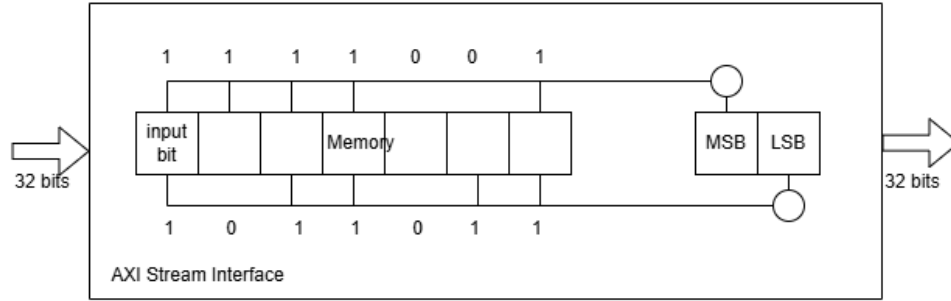


Figure 4.4: Final encoder hardware architecture: 32-bit AXI input register, bit-serial core, 64-bit output buffer, two-word AXI output

### 4.10.2 Stream protocol design rules

The AXI4-Stream handshake, a transfer occurs exactly on a rising clock edge with **TVALID** and **TREADY** simultaneously high was adopted with the following project rules, each traceable to a defect found during verification: producers assert **TVALID** whenever data is available and never gate it on **TREADY** (gating creates a standoff deadlock); counters and monitors qualify every event on the full handshake, never on **TREADY** alone; **TLAST** frames packets, and for the encoder is emitted only on the second of the two output words; HLS block-level **ap\_start** is registered high one cycle after reset release; and data buses are treated as undefined ('bx') whenever **TVALID** is low, which the HLS-generated combinational output multiplexers exploit intentionally. The decoder's input **TREADY** provides natural backpressure to the encoder; an optional AXI-Stream FIFO between the blocks was designed for buffering experiments.

## 4.11 Signal Flow and Timing

End-to-end hardware timing of the coding chain, established later by measurement (Chapter 7): encoder latency 19 cycles, interface overhead approximately 14 cycles, decoder latency 51 cycles; 84 cycles from first accepted input word to first decoded output word at 100 MHz (840 ns). The decoder dominates both latency and throughput.

# Chapter 5

## Implementation

### 5.1 MATLAB Implementation

#### 5.1.1 Code organization

The MATLAB implementation evolved through two phases mirroring the repository layout. Phase 01 realizes the complete AWGN packet loop: CSV-driven payload input in an AXI-Stream-like format (`TDATA`, `TKEEP`, `TLAST` records), packet construction, encoding, pulse shaping, modulation, AWGN, demodulation, preamble detection, payload extraction, soft Viterbi decoding, and CSV output, with both single-packet (`test_complete_system.m`) and multi-packet (`test_multipacket_system.m`) drivers. Phase 02 replaces library-dependent coding with the custom `convEncoder.m` and `viterbi_soft_decoder.m`, adds `channel_model.m` (GSM HT plus AWGN), and introduces the equalization subsystem (`lms_equalizer_complex.m`, `gmsk_modulate_burst_ref.m`, `preamble_sample_indices.m`, `demod_aligned_burst.m`, `sweep_equalizer_params.m`), orchestrated by `main_system.m`. A dependency-ordered build plan (enumerations  $\rightarrow$  waveform  $\rightarrow$  burst container  $\rightarrow$  I/O  $\rightarrow$  coding/framing  $\rightarrow$  shaping/modulation  $\rightarrow$  TX top  $\rightarrow$  RX chain  $\rightarrow$  integration scripts) was derived from static analysis of the legacy codebase before modification.

#### 5.1.2 Legacy code analysis

The inherited `TxModulator` System object was analysed line-by-line before being reworked. Two latent defects were identified: the encoder object was invoked twice per block in the processing chain (almost certainly erroneous), and a precomputed output-size value intended for preallocation was computed but never used, leaving the output vector to grow by concatenation. Both observations informed the rewritten transmit chain.

### 5.1.3 Soft-decision Viterbi reference decoder

The reference decoder accepts interleaved floating-point LLR pairs, reshapes them into per-symbol rows, reconstructs for every trellis transition the expected encoder output by re-encoding the predecessor state, computes correlation branch metrics, performs the 64-state ACS with survivor recording, and applies sliding-window traceback of depth  $T_B$  (32–40), emitting one bit per stage after window fill. Two implementation defects were found and corrected in the final-flush logic: an index misalignment between the sliding-phase and flush-phase output positions, and an output-trimming statement that discarded the very bits the flush had just recovered. Floating-point arithmetic was retained deliberately: the reference bounds attainable performance, and quantization effects belong to the hardware model.

### 5.1.4 Simulation drivers and performance engineering

BER experimentation is served by both a script driver (fixed sweep, byte-level comparison through the full receiver) and a parameterized function driver returning a results structure (own silent channel application, bit-level comparison, target-SNR interpolation); their differing error-counter initializations (1 versus 0) and stopping parameters were documented explicitly, since a counter initialized at 1 biases high-SNR BER upward. Monte Carlo throughput of the sweep was improved by profiling: the dominant cost was per-iteration reconstruction of static objects inside the receiver call, removed by **persistent** caching, alongside suppression of per-burst console output and conditional CSV writing. A subsequent request reverted the receiver-side changes to preserve the original file, retaining the driver-side improvements, version control making such selective rollback routine.

## 5.2 HLS Implementation of the Convolutional Encoder

Two encoder generations were implemented. The first, bit-serial variant streamed one bit per transaction; it contained a one-line defect; the packet’s **last** flag written into the *data* field of the second output beat (`out_pkt.data = in_pkt.last`) caught by comparison against MATLAB, and an II violation from two writes per iteration to one stream, resolved by widening the output to `ap_axis<2>` and packing both coded bits into a single beat. The second, final variant (`convEncoder_Seq`) realizes the architecture of Section 4.10: one 32-bit word in, a 32-iteration bit-serial loop, two 32-bit words out, with the 6-bit state held in a **static** variable to preserve continuity across words and **TLAST** forwarded on the second beat only. The generated RTL was reverse-engineered to confirm the mapping: a two-level hierarchy (top-level AXI/FSM wrapper and a loop-pipeline submodule), a four-

state controller (capture input and TLAST; run pipeline via `ap_start/ap_ready`; emit lower word; emit upper word, commit state, forward TLAST), and the expected XOR network. Two pragma-induced defects dominated co-simulation debugging and are analysed in Section 6.9; the corrected interface specification is:

```
#pragma HLS INTERFACE axis port=s_axis register_mode=off
#pragma HLS INTERFACE axis port=m_axis register_mode=off
// No PIPELINE pragma at the top level; no UNROLL on the
// state-carrying encoding loop.
```

## 5.3 HLS Implementation of the Viterbi Decoder

### 5.3.1 From studied RTL to HLS

Implementation began from a working 4-state ( $K = 3$ ) SystemVerilog decoder, whose seven modules (`acs_rad2`, `vit_bmu`, `vit_acs`, `vit_fsm`, `tbu_dec`, `vit_tbu`, `vit_top`) were studied to extract the architecture: registered BMU computing  $\pm g_0 \pm g_1$ ; four ACS butterflies with hardwired trellis connections; a seven-state control FSM scheduling four rotating 16-symbol survivor banks into 32-symbol traceback windows served by two parallel traceback decoders. A direct SystemVerilog-to-Verilog conversion was considered and abandoned; instead the design was re-expressed algorithmically in HLS C++, with clocked processes and handshakes replaced by loops and `hls::stream` interfaces, the two traceback decoders unified into one window-based routine, the FSM replaced by loop counters with bank-done detection, and control parameters (`info_seq_len`, `init_neg`) exposed over AXI-Lite. A critical review of this conversion against the RTL identified five mismatches most seriously a sequence-length off-by-two into the traceback unit (+1 in RTL versus  $-1$  in HLS), unpreserved BMU/ACS register timing, simplified traceback scheduling, stricter input-gap assumptions, and a header comment contradicting the implemented tail-injection behaviour, establishing that HLS conversion preserves the algorithm but not the cycle behaviour unless explicitly forced.

### 5.3.2 The 64-state production decoder

The production decoder (`viterbi_decoder`) targets  $K = 7$ : 64 states, Hamming metrics, traceback depth 32, one 32-bit AXI word of 16 packed 2-bit symbols in, 16 decoded bits (zero-extended to 32) out. Its five blocks map to hardware as follows: the BMU synthesizes a three-level XOR parity tree per generator (approximately 6 LUTs) plus a 2-bit Hamming unit; the 64-way ACS loop is fully unrolled with the input bit recovered as `ns[5]` and 9-bit saturating adders preventing metric wrap; survivor memory `survivor[32][64]` is completely partitioned in the state dimension so all 64 bits write

in one cycle; the minimum-metric search unrolls into a six-level comparator tree; and traceback is a 16-step sequential walk. The directive set and its rationale:

```
#pragma HLS PIPELINE II=1           // one trellis stage per clock
#pragma HLS ARRAY_PARTITION variable=pm complete dim=1
#pragma HLS ARRAY_PARTITION variable=survivor complete dim=2
#pragma HLS INLINE                  // helpers into ACS datapath
#pragma HLS INTERFACE ap_ctrl_none port=return
#pragma HLS INTERFACE axis port=symbols_in
#pragma HLS INTERFACE axis port=bits_out
```

Complete partitioning of the path-metric array is what permits the 64 ACS updates to read and write in a single cycle; `ap_ctrl_none` removes block-level handshaking so the core is purely stream-driven.

### 5.3.3 Parameterized and soft-decision variants

Two further decoder implementations broadened the design space. A parameterized variant (`viterbi_k7`) generates all four trellis tables (`NEXT_STATE`, `ENC_OUT`, `PRED_STATE`, `PRED_ENC_OUT`, each  $64 \times 2$ ) as `constexpr` arrays at compile time, eliminating both run-time cost and manual wiring, and adds subtract-minimum path-metric normalization and an `II=1` pipelined sliding traceback with survivor memory bound to a single 18K BRAM; its Tcl-scripted flow targets 200 MHz with expected resources of 2000–4000 LUTs. A soft-decision variant (`convDecoder`) accepts quantized soft inputs with fully unrolled 64-state ACS and BRAM traceback at `II=2` (two soft reads per trellis step), designed as the hardware successor once soft metrics are carried into the FPGA.

## 5.4 Memory Organization

Memory design decisions across the hardware blocks: path metrics in fully partitioned registers (parallel ACS access); survivor storage in BRAM, dimension-partitioned for single-cycle row writes, organized as rotating banks in the streaming variants; GMSK mapper sine/cosine tables in BRAM via `BIND_STORAGE`; trellis connectivity in `constexpr` tables synthesized to logic rather than ROM; and all arrays statically allocated (dynamic allocation is unsynthesizable and was excluded by construction).

## 5.5 HLS Implementation of the GMSK Mapper

The mapper port comprises five files: parameter/type header (`MAX_LEN=1024`), implementation, precomputed 512-entry Q15 LUT header, self-checking testbench, and a Python LUT generator replicating MATLAB’s quantization exactly. The implementation pipelines

the sample loop at  $\Pi = 1$ , reproduces the asymmetric saturation ( $\pm 65535$  threshold,  $\pm 32768$  clamp) in an inlined helper, and corrects the C++ `%` operator to MATLAB `mod` semantics for negative accumulator values, the single semantic trap of the port. Six C-simulation tests (all-zero, positive/negative ramps with LUT wrap, both saturation directions, alternating  $\pm 17$  stress) pass against an inline golden model.

## 5.6 On-Chip Measurement Infrastructure

The hardware throughput bench (Section 6.12) required two synthesizable support blocks. A 32-bit Galois LFSR (`lfsr32.sv`) replaces the unsynthesizable `$random`: the Galois form was chosen over Fibonacci because its feedback XORs are applied in parallel positions rather than chained on the critical path; the polynomial mask `32'h8020_0003` encodes  $x^{32} + x^{22} + x^2 + x + 1$ ; the update is a single shift-and-conditional-XOR line synthesizing to one LUT layer; the seed is non-zero by necessity (the all-zero LFSR state is absorbing); and the enable is the AXI handshake itself, freezing the generator under backpressure. The bench top (`throughput_bench_top.sv`) instantiates LFSR, encoder and decoder; ties `TVALID` to reset release for gap-free stimulus; frames `TLAST` every 64 words; registers `ap_start` one cycle after reset; drains the decoder with `TREADY=1`; sizes its warmup and window counters with `$clog2` while keeping 64-bit accumulators; and deliberately performs no division on chip a 64-bit divider would cost thousands of LUTs and threaten timing, so only the raw counters are exposed and the quotient is computed off-chip.

# Chapter 6

## Verification and Experimental Work

This chapter documents the experimental programme in the order in which it was executed, including the failed experiments and the debugging investigations, since these constitute a substantial fraction of the engineering work.

### 6.1 MATLAB Functional Verification

The Phase 01 loop was verified end-to-end: payload bytes read from CSV, transmitted through the full chain under AWGN, and recovered byte-exactly at the receiver, for both single-packet and multi-packet streams. A small side experiment verified the coding subsystem in isolation over a binary symmetric channel: after correcting two script defects (an undefined flip probability, and a channel that was never actually applied so that `rx = coded`), the decoder reduced a raw flip probability of  $p = 0.05$  over 1152 bits to a post-decoding BER of  $1.13 \times 10^{-2}$  (13 errors), confirming corrective behaviour in the presence of genuine channel errors. A transmit-side spectral measurement was added to the multipacket driver: an FFT-based estimate of the transmitted PSD yielded a 99% occupied bandwidth of 613.22 kHz and a  $-3$  dB bandwidth of 500.06 kHz at the simulated sample rate, consistent with the compact spectrum expected of  $BT = 0.35$  GMSK.

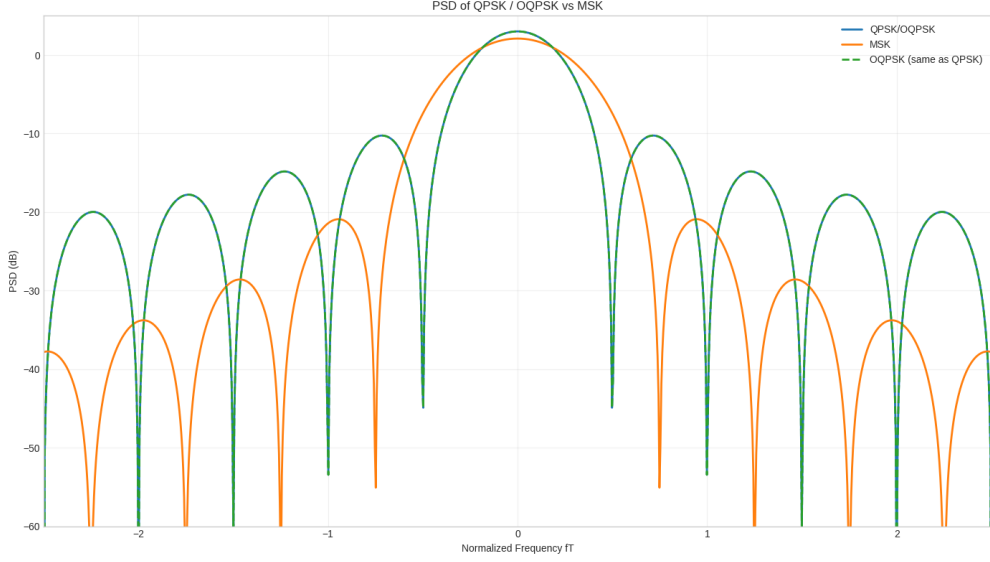


Figure 6.1: TX Power Spectral Density Plot

## 6.2 BER Methodology Development

The first AWGN sweep (100 trials per point,  $\approx 25,600$  bits) exposed the statistics of rare events: above approximately 9 dB no errors were observed, the estimated BER became exactly zero, and the log-scale curve silently vanished. The resolution was methodological, not computational: to resolve BER near  $10^{-5}$ , on the order of  $10^6$  bits are required so that the expected error count ( $\approx 10$ ) is observable. The adopted methodology therefore runs each SNR point until either a minimum error count (100–1000 depending on driver) or a maximum bit budget ( $10^5$ – $2 \times 10^6$ ) is reached; stores NaN rather than zero for error-free points so plotting remains continuous; terminates the sweep after three consecutive error-free points; and locates the  $10^{-5}$  crossing by log-domain interpolation between bracketing points rather than by inspection. The axis convention was converted from  $E_b/N_0$  to SNR, replacing the rate- and oversampling-dependent conversion with the direct definition  $P_n = P_s/10^{\text{SNR}_{\text{dB}}/10}$ . Plot presentation (log-domain crosshair segments drawn to the intersection only) was standardized after `yline/xline` proved unreliable on logarithmic axes.

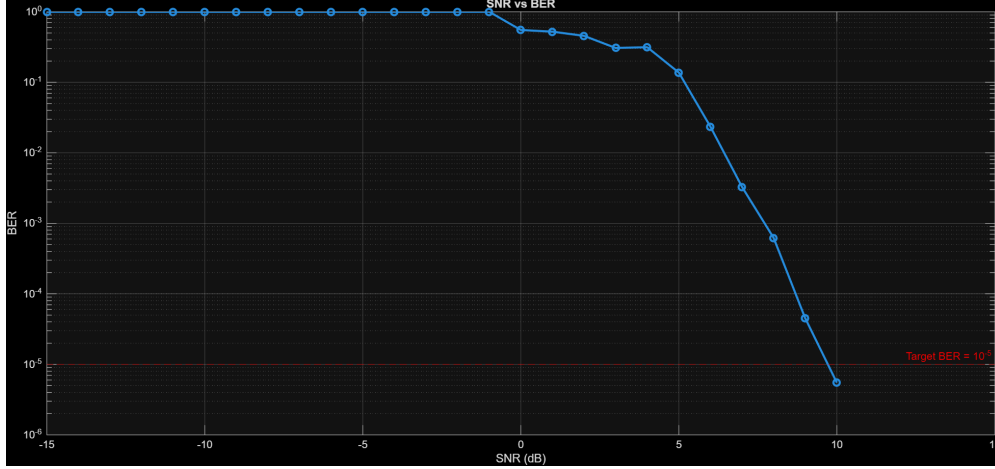


Figure 6.2: AWGN BER versus SNR with  $10^{-5}$  crossing annotation

The AWGN curve exhibits the expected coded waterfall: BER near unity at low SNR (missed preamble detections charge whole payloads as errors), rapid improvement through 4–7 dB, and error-free operation beyond approximately 8 dB.

### 6.3 Verification of the Soft-Decision Reference Decoder

The sliding-window decoder was verified against the encoder over parameterized frame lengths; the missing-tail phenomenon ( $N - T_B + 1$  output bits) was reproduced, diagnosed as inherent to unterminated sliding traceback, and the flush-logic defects (index misalignment; post-flush trimming) corrected. The end-of-frame analysis of Section 4.8 was validated by observing tail-bit reliability degradation without termination.

### 6.4 Rayleigh-Stage Experiments

The modular flat-Rayleigh path (channel, LS single-tap estimator, one-tap equalizer, BER driver) was implemented to mirror the AWGN flow, with training positions derived from the bit map mask so that the estimator respects the interleaved frame layout.

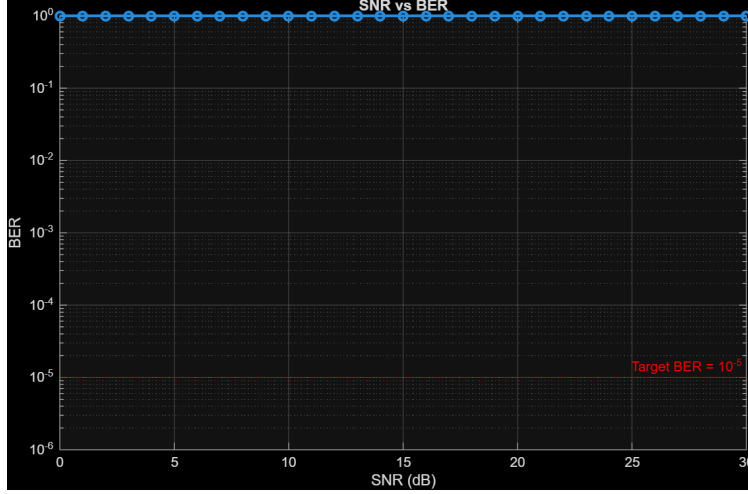


Figure 6.3: flat Rayleigh without equalization

## 6.5 GSM Hilly-Terrain Integration and the BER = 1 Investigation

Integration of `gsmHTx12c1` produced an immediate null result:  $\text{BER} = 1$  at every SNR. The investigation proceeded in two steps. First, a stimulus defect was found the computed noise vector was never added (`rxSignal = fadedSignal`) which was corrected but did not explain the failure. Second, the true cause was architectural: the receiver, designed under AWGN assumptions (no multipath, no amplitude fading, no random phase), was presented with a 12-tap time-varying channel; the distorted phase trajectories rendered the soft metrics meaningless and decoding collapsed. Beyond the specific fixes (channel reset per trial, power normalization, reduced Doppler during bring-up), the enduring lesson was methodological:  $\text{BER} = 1$  (as opposed to  $\approx 0.5$ ) signals a structurally broken link, and channel realism must be increased in stages so each new impairment is attributable.

## 6.6 Equalizer Development Experiments

The first LMS integration attempts failed to converge, and the ensuing investigation the longest of the project, isolated four independent defects whose individual symptoms overlapped:

1. **Reference-domain mismatch.** The equalizer compared complex GMSK samples against a real NRZ  $\pm 1$  reference; the error's large imaginary component drove the taps away from any useful solution. Resolved by the bit-exact complex reference of Section 4.9.
2. **Integer-type defect.** Training bits stored as `uint8` made the mapping  $2b - 1$  saturate rather than produce  $-1$ ; resolved by explicit conversion to `double`.

3. **Burst-timing offset.** Burst extraction ignored the demodulator group delay, starting 16 samples early and misaligning every training sample (Section 6.8).
4. **Training-reference contamination.** Even when aligned, mid-burst training samples are functions of adjacent unknown payload bits through the modulation memory; adaptation on them chases an unrealizable target.

With the NLMS equalizer verified in isolation (a synthetic-channel test with dense training converged with a 3.15 dB MSE improvement; sparse-training tests showed strong sensitivity to  $\mu$  and filter length), a decision delay was introduced and a systematic parameter sweep over  $(\mu, \text{eq\_len}, D)$  was run with *byte errors after decoding* as the objective function a deliberate replacement of the misleading MSE criterion. The best sweep configuration ( $\mu = 0.01$ , 13 taps,  $D = 12$ ) still yielded 31 byte errors in 34, which was the decisive evidence that parameter tuning could not repair a structural defect. The subsequent root-cause pass produced the two final fixes; group-delay-corrected burst extraction, and retraining on the contiguous preamble, after which the receiver decoded **34 of 34 payload bytes without error at 24 dB SNR over the hilly-terrain channel**. Notably, the equalizer’s convergence monitor still reported “NOT CONVERGED”: the stale MSE heuristic contradicted a perfect decode, reinforcing the choice of end-to-end error counts as the only trustworthy objective.

## 6.7 Realistic-Channel BER Campaign

The three-case BER campaign (AWGN; HT without equalization; HT with NLMS equalization) was then executed with the stopping-rule methodology. Under HT with equalization the measured BER at 15 dB was  $1.0 \times 10^{-1}$ , and the  $10^{-5}$  target was reached at around 24 dB; false preamble detections were identified as the dominant low-SNR failure mechanism. These results are analysed in Chapter 7.

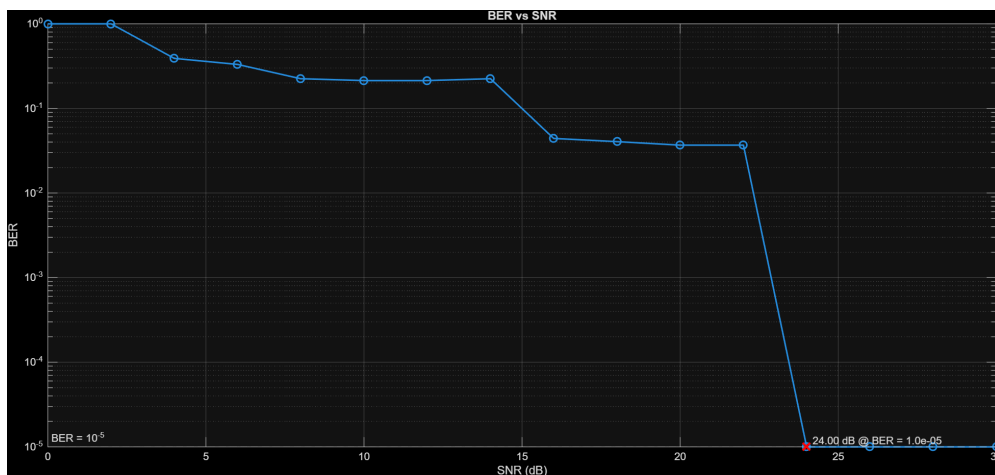


Figure 6.4: GSM HT with NLMS equalization

## 6.8 The Group-Delay Alignment Investigations

Three superficially unrelated receiver failures shared one root cause; inconsistent accounting of the 16-sample filter group delay across the sample, symbol and LLR index domains:

1. **616 versus 624.** Demodulating an already-extracted 624-symbol burst re-applied the demodulator's edge trimming, losing 8 symbols and crashing payload indexing (`llrBurst(1:624)` on a 616-element vector). The working BER scripts were immune because they demodulate the full waveform before extraction. Resolution: a dedicated aligned-burst demodulation path (`demod_aligned_burst.m`); alternatives (guard-sample extension, defensive length checks) were evaluated and documented.
2. **Sixteen-samples-early extraction.** The symbol-to-sample conversion omitted  $g_d$ , since LLR index 1 corresponds to sample 17; corrected to  $n_{samp} = g_d + (n_{sym} - 1) \cdot \text{SPS} + 1$ .
3. **Preamble index 4.** A suspected detector fault was cleared: the offset is the deterministic consequence of filter delay (correlation 1.188 at the correct position).

The consolidated lesson carry group delay explicitly through every index-domain conversion is recorded as a design rule in Chapter 8.

## 6.9 HLS Encoder Verification

The encoder was verified at three levels. C simulation ran a ten-case golden-model suite (all-zeros, all-ones, impulses at both word boundaries, alternating patterns, canary word, multi-word state persistence, 32 walking-ones subtests, random burst): 41 test vectors in total. RTL co-simulation then failed twice in instructive ways. In the first failure only 1 of 41 vectors passed, with every result displaced by one transaction; the causes were the top-level PIPELINE pragma (the co-simulation wrapper returned control after one cycle, before the output stream was written) and the UNROLL pragma on the state-carrying loop (a 32-deep combinational dependency chain). In the second failure, with C simulation already at 41/41, the RTL still showed the one-transaction shift; the log revealed automatically inserted AXI register slices adding one cycle of output latency beyond `ap_done`, cured by `register_mode=off` on both stream ports; the per-transaction progress indicator (33% stuck versus 100%) provided the confirming evidence. Final status: 41/41 in both C and RTL co-simulation. File-based RTL verification followed: a 10,000-bit input file packed LSB-first into 313 words (with zero-padding of the final partial word) produced 626 output words 20,032 coded bits, the 32-bit excess over  $2 \times 10^4$  exactly attributable to the padding.

## 6.10 HLS Decoder Verification and the Block-Boundary Incident

The decoder algorithm was first validated standalone (no HLS constructs): all 11 tests with tail flushing passed; without flushing 12 of 13 passed, the single failure being the documented last-bit truncation effect. The full BER testbench then generated 10,000 reproducible random bits (`srand(42)`), encoded them with a bit-exact software reference, packed 16 symbols per word MSB-first, streamed 625 blocks through the decoder, and compared outputs with per-position statistics separating core bits from the final  $K - 1$  tail positions. The first full run returned a BER of 40% with systematic failures from block 2 onward the most instructive single defect of the project. The root cause lay in the *testbench*, not the decoder: the reference encoder ran once over all 10,000 bits, carrying its shift-register state across block boundaries, while the decoder initializes every block assuming state 0; block 0 therefore decoded, block 1 coincidentally converged, and every subsequent block failed. Re-encoding per block from the zero state restored agreement: **0 errors in 10,000 bits**. The incident generalizes to a design rule, encoder and decoder must share the same state assumption at every block boundary and foreshadows the zero-tail termination planned for the framed system. The scope of this verification was recorded honestly: it exercises the ideal channel only (no noise, fading, timing error, soft decisions, or cross-block streaming), so its BER of zero is a correctness statement, not a performance claim. An attempt to package the verified decoder as a Vivado IP crashed the tool (`alloc: invalid block, EXCEPTION_BREAKPOINT`, no usable stack trace); since C simulation had passed, the fault was attributed to the tool/environment rather than the design, and the hardware flow proceeded by adding the generated RTL sources directly to the Vivado project.

## 6.11 RTL System Simulation: Loopback Testbenches

A sequence of SystemVerilog loopback testbenches connected the generated encoder and decoder RTL for system-level protocol verification and throughput measurement, instrumented with cycle counters, per-stream transaction counters, first-in/first-out latency capture, steady-state and total throughput computation, utilization and effective-II estimation, an optional AXI FIFO (`USE_FIFO`), a watchdog, and VCD waveform dumping. The workload was 1000 random 32-bit words (32,000 information bits; 2000 encoded words; 2000 decoded words of 16 valid bits each). The testbench campaign surfaced and resolved an unusually rich defect set, summarized in Table 6.1. Each entry was diagnosed from waveforms or simulator warnings and re-verified after the fix. After the corrections, the loopback measured a pipeline latency of 84 cycles and steady throughput of approximately 0.327 bits/cycle (32.7 Mb/s at 100 MHz), with the decoder's observed initiation

Table 6.1: Defects found and fixed during RTL system simulation.

| Symptom                                    | Root cause                                                                   | Fix                                                    |
|--------------------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------|
| Zero throughput; no encoder input activity | TVALID gated on TREADY (handshake deadlock)                                  | Producer asserts TVALID unconditionally                |
| Zero throughput after 100 cycles           | run 1000ns for a $\sim 2.2$ ms workload                                      | run -all; \$finish self-termination                    |
| 1998 of 2000 outputs; watchdog fires       | initial+always_ff multiple drivers on counters (race)                        | All resets moved into the always_ff async-reset clause |
| Last transaction lost                      | TREADY-only sampling instead of full-handshake edge sampling                 | Sample on TVALID && TREADY at posedge                  |
| Throughput overstated 2 $\times$           | output_bits += 32 though only 16 bits valid per decoder word                 | += 16                                                  |
| 32-bit port driven serially                | Testbench assumed bit-serial interface; DUT expects full words per handshake | Drive complete 32-bit words                            |
| Encoder idle 10 cycles                     | ap_start asserted late (200 ps race)                                         | Registered ap_start from reset                         |
| Undefined TLAST                            | Signal left undriven ('bx)                                                   | Driven explicitly (0 for continuous streams)           |
| Red ('bx) data buses in waveform           | Intentional HLS don't-care outside TVALID                                    | No fix required; qualify data on TVALID                |
| Watchdog margin too small                  | Drain time of final words exceeded limit                                     | Limit raised to 500,000 cycles                         |

interval ( $\approx 48.9$  cycles/word) in close agreement with the HLS report (51). An early file-driven run, executed before the final-handshake fix, recorded 4.99 Mb/s with one lost input transaction retained in the record as an example of how measurement infrastructure defects masquerade as performance results. An external simulation attempt with Icarus Verilog could not run (tool unavailable in that environment) and was substituted by a scripted structural audit of the testbench before the Vivado runs.

## 6.12 Hardware Validation on the Zybo Z7-10

The measurement design of Section 5.6 was synthesized and implemented in Vivado with `throughput_bench_top` as top module. Board integration required its own debugging round: the 125 MHz oscillator on pin K17 was constrained correctly after clarifying that `create_clock` describes rather than sets frequency (an MMCM being the correct route to other frequencies); a debug-hub property applied in the XDC before any ILA existed produced a “no ChipScope cores” error and was relocated to post-insertion; a programmed bitstream lacking the ILA that its probe file expected (“no supported soft debug cores”) was cured by regenerating matched `.bit/.ltx` pairs from the same implementation run; and the debug hub clock was corrected from its 300 MHz default to the true 125 MHz. The ILA (1024-sample depth, triggered on `measurement_done`) then captured the frozen counters. At a 50 MHz system clock, the capture read `cycs_q` = 100,000,000 and `bits_q` = 65,306,144, i.e.

$$\frac{65,306,144 \times 50 \times 10^6}{100 \times 10^6} = 32,653,072 \text{ b/s} \approx 32.65 \text{ Mb/s},$$

in agreement with RTL simulation.

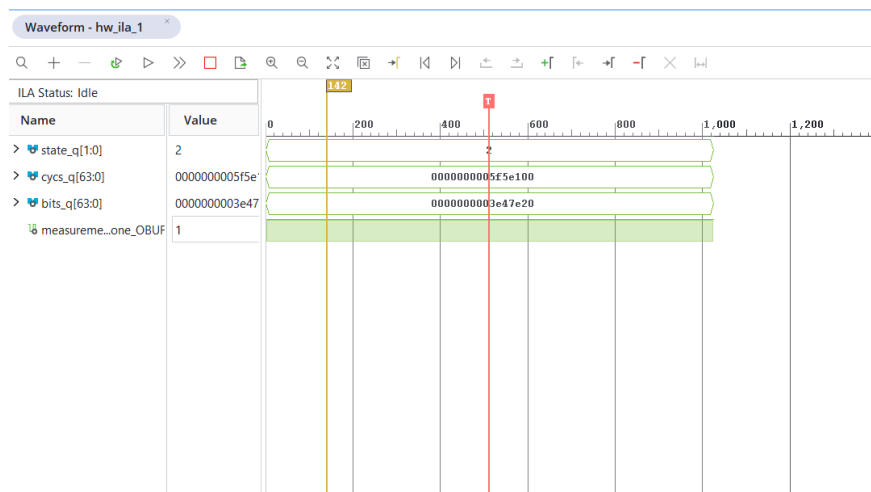


Figure 6.5: ILA capture on the Zybo Z7-10 showing the frozen counters `bits_q` and `cycs_q` after triggering on `measurement_done`.



Figure 6.6: Zybo Z7-10 board running the throughput measurement design.

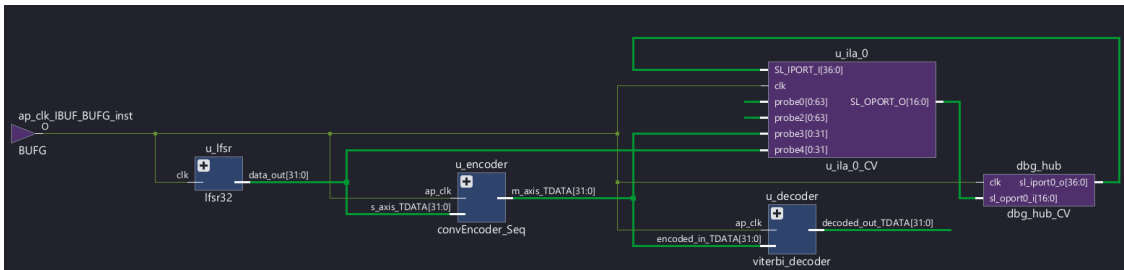


Figure 6.7: Vivado synthesis report: Dataflow Design throughput\_bench\_top.

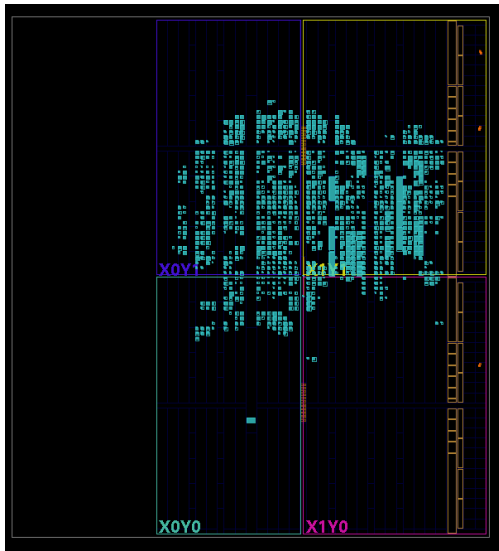


Figure 6.8: Vivado implementation (post-place-and-route) utilization report.

| Setup                                               | Hold                                            | Pulse Width                                             |
|-----------------------------------------------------|-------------------------------------------------|---------------------------------------------------------|
| Worst Negative Slack (WNS) <a href="#">1.632 ns</a> | Worst Hold Slack (WHS) <a href="#">0.051 ns</a> | Worst Pulse Width Slack (WPWS) <a href="#">2.750 ns</a> |
| Total Negative Slack (TNS) 0.000 ns                 | Total Hold Slack (THS) 0.000 ns                 | Total Pulse Width Negative Slack (TPWS) 0.000 ns        |
| Number of Failing Endpoints 0                       | Number of Failing Endpoints 0                   | Number of Failing Endpoints 0                           |
| Total Number of Endpoints 7042                      | Total Number of Endpoints 7026                  | Total Number of Endpoints 4239                          |
| All user specified timing constraints are met.      |                                                 |                                                         |

Figure 6.9: Post-implementation timing summary showing worst negative slack at the target clock frequency.

## 6.13 Validation Strategy Summary

Every performance claim in Chapter 7 is traceable to one of: a stopping-rule Monte Carlo simulation (BER), a counter-instrumented RTL simulation (latency, II, simulated throughput), or an ILA-captured on-chip measurement (hardware throughput). Every functional claim is traceable to a golden-model comparison at C or RTL level. Defects found by each layer were, without exception, invisible to the layer above it the strongest available justification for the layered strategy.

# Chapter 7

## Results and Discussion

### 7.1 Communication Performance

Table 7.1: Summary of communication performance results.

| Experiment                   | Condition                           | Result                                                               |
|------------------------------|-------------------------------------|----------------------------------------------------------------------|
| AWGN BER sweep               | Full modem, soft decoding           | Waterfall 4–7 dB; error-free beyond $\approx 8$ dB                   |
| TX occupied bandwidth        | FFT of transmit signal              | 613.22 kHz (99%); 500.06 kHz ( $-3$ dB)                              |
| BSC coding test              | $p = 0.05$ , 1152 bits              | Post-decoding BER $1.13 \times 10^{-2}$                              |
| GSM HT, no equalization      | AWGN-designed receiver              | BER = 1 (structural failure; motivated equalization)                 |
| GSM HT + NLMS, single packet | 24 dB SNR, preamble training        | 34/34 bytes correct (0 errors)                                       |
| GSM HT + NLMS, Monte Carlo   | 15 dB SNR                           | BER $\approx 1.0 \times 10^{-1}$ ; $10^{-5}$ reached at around 24 dB |
| Equalizer sweep (pre-fix)    | best $(\mu, L, D) = (0.01, 13, 12)$ | 31/34 byte errors; tuning insufficient                               |
| NLMS sanity test             | synthetic channel, dense training   | Converged; 3.15 dB MSE improvement                                   |

The AWGN result confirms that the complete chain including framing, synchronization and soft decoding realizes the expected coded-GMSK behaviour; the soft-decision design choice is consistent with the  $\approx 2$  dB advantage over hard decisions reported in the literature [2, 12], although a controlled hard-versus-soft comparison on this modem was not run. The hilly-terrain results warrant careful interpretation. The single-packet result at 24 dB demonstrates that the receiver architecture coarse synchronization, group-delay-exact burst extraction, bit-exact preamble reference, NLMS with decision delay is *correct*: the severe 12-tap channel is equalized to error-free decoding. The Monte Carlo campaign

confirms that the architecture is also *robust* at moderate-to-high SNR: the  $10^{-5}$  target is reached at approximately 24 dB, with false preamble detections remaining the dominant error mechanism at low SNR. The residual mechanisms (detection threshold behaviour in noise; adaptation on 48 preamble bits only; no interleaving against fade bursts) are individually addressable and are carried into future work rather than obscured.

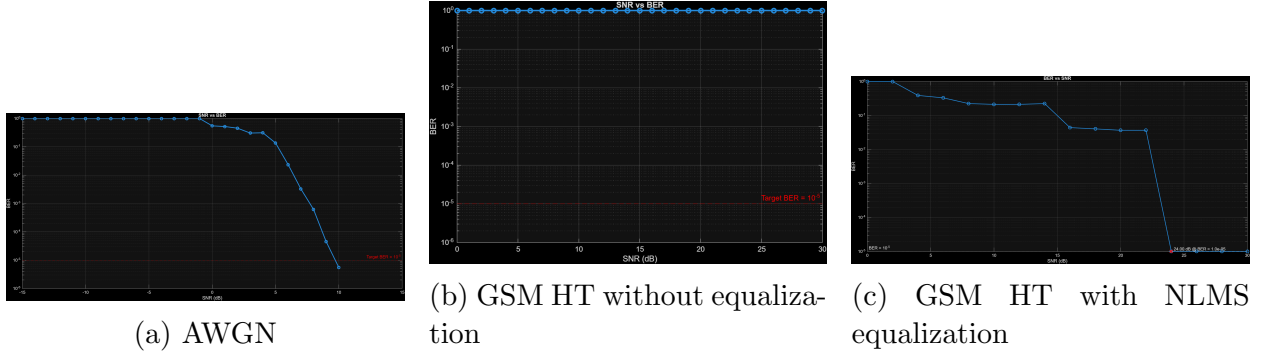


Figure 7.1: BER comparison across channel conditions: (a) AWGN, (b) GSM HT without equalization, and (c) GSM HT with NLMS equalization.

## 7.2 Hardware Performance

Table 7.2: HLS synthesis results for the hardware blocks.

| Module          | Latency (cycles) | FF  | LUT    | Est. clock (ns) |
|-----------------|------------------|-----|--------|-----------------|
| convEncoder_Seq | 19               | 156 | 247    | 2.018           |
| viterbi_decoder | 51               | 804 | 10,214 | 6.930           |

Table 7.3: Measured system performance of the encoder–decoder chain.

| Metric                                | Value                                                   |
|---------------------------------------|---------------------------------------------------------|
| Pipeline latency (100 MHz RTL sim)    | 84 cycles = 840 ns (19 enc + $\approx$ 14 AXI + 51 dec) |
| Steady throughput (RTL sim, 100 MHz)  | $\approx$ 0.327 bits/cycle $\approx$ 32.7 Mb/s          |
| Decoder effective II (measured / HLS) | $\approx$ 48.9 / 51 cycles per 16-bit word              |
| On-board throughput (Zybo, 50 MHz)    | 32.65 Mb/s (ILA: 65,306,144 bits / $10^8$ cycles)       |
| Bottleneck                            | Viterbi decoder traceback block                         |

Three observations follow. First, simulation and hardware agree to within measurement resolution ( $\approx 32\text{--}33$  Mb/s), which validates the entire measurement chain; counters, handshake qualification, warmup windowing, ILA capture. Second, the throughput is decoder-limited: the encoder’s 19-cycle, II-free streaming is throttled by the decoder’s  $\approx 50$ -cycle block cadence, so per-block traceback, not the parallel ACS array, is the target of any optimization; the parameterized `viterbi_k7` variant, with pipelined sliding traceback at  $\text{II} = 1$ , is the designed successor. Third, the measured figures are *internal processing throughput*, not a wireless data rate: an early, flawed measurement of 2.09 Gb/s (from a misconfigured testbench) prompted an explicit analysis distinguishing interface bandwidth, processing throughput, and over-the-air rate, the last being bounded by symbol rate and channel bandwidth ( $\sim 270$  kb/s-class for a GSM-parameter GMSK carrier) rather than by the FPGA.

### 7.3 Design Trade-offs Revisited

The principal trade-offs, with outcomes: traceback over register exchange traded latency for an order-of-magnitude storage saving, and the measured 84-cycle latency confirms the cost is modest at system level; the bit-serial encoder behind a 32-bit interface traded peak parallelism for minimal area (247 LUTs) while pipelining preserved throughput far above the decoder’s ceiling vindicating the rejection of the dual-core proposal; full ACS unrolling with complete metric partitioning bought one-stage-per-cycle trellis processing at the price of the decoder’s 10k-LUT footprint; and floating-point MATLAB references traded fidelity to the eventual fixed-point hardware for an unambiguous performance bound and clean layer separation.

### 7.4 Comparison with Theoretical Expectations

Where theory makes predictions, the measurements conform: the coded AWGN waterfall appears in the expected SNR region for a  $d_{free} = 10$  rate-1/2 code with soft decisions, consistent with published union-bound and simulation results for this code [9, 13]; the traceback depth of 32 sits just below the  $5K = 35$  rule with the observed mild tail-bit degradation; the unequalized HT channel destroys an AWGN receiver exactly as multipath theory predicts; and the decoder II observed in simulation (48.9) matches the HLS schedule (51) within pipeline-overlap effects. The occupied-bandwidth measurement (613 kHz at  $4 \times 270.833$  kHz sampling) is consistent with published GMSK spectra for  $BT$  near 0.3–0.35 [7].

## 7.5 Engineering Observations

Beyond the numbers, the project yields transferable observations. (i) In HLS, pragmas are architecture, not annotation: two pragmas silently converted a correct encoder into one that failed co-simulation, and interface registering altered protocol timing. (ii) Verification infrastructure is itself a design artefact with its own defect density; roughly half of all hardware-side defects were in testbenches and measurement logic, not in the DUTs. (iii) Cross-domain index bookkeeping (samples, symbols, LLRs) deserves the same rigour as datapath design; a single 16-sample constant produced three distinct system failures. (iv) Optimization objectives must be end-to-end: MSE said the equalizer had failed while the decoder delivered a perfect payload. (v) Structured specification of generated code, followed by audit, is an effective but not sufficient practice, every generated artefact that entered the flow required at least one substantive correction.

## 7.6 Limitations

The current results carry the following limitations, stated candidly. The hardware decoder is hard-decision and block-oriented, forfeiting  $\approx 2$  dB and requiring per-block state alignment; the framed system's zero-tail termination is designed but not yet implemented in hardware. The hardware BER has not been characterized under noisy channels, the hardware testbench is an ideal-channel correctness check. The HT-channel receiver reaches the  $10^{-5}$  target only at a comparatively high SNR of approximately 24 dB, with false detections dominant at low SNR; lowering this operating point remains an open task. The full GMSK signal path (filter, mapper beyond C-sim, demodulator, synchronizer, equalizer) is not in hardware. Fixed-point effects on the receiver algorithms are unquantified. Finally, the on-chip bit counter increments by 32 per decoder word while 16 bits are valid; the reported hardware throughput is consistent with the valid-bit interpretation and with simulation, but the counter convention should be unified in the next revision.

# Chapter 8

## Conclusion

### 8.1 Achievements

The project set out to design, model, evaluate and partially implement in hardware a complete GMSK burst modem, and achieved: a fully functional MATLAB modem with framing, coding, pulse shaping, modulation, synchronization, soft-decision decoding and CSV-based I/O; a defensible BER methodology with stopping rules and target interpolation; evaluation under AWGN and under the standardized COST 207 hilly-terrain channel; a receiver enhancement programme (LS estimation, complex NLMS with decision delay, preamble training, group-delay-exact synchronization) culminating in error-free decoding over the HT channel and attainment of the  $10^{-5}$  BER target at approximately 24 dB; HLS implementations of the encoder, three Viterbi decoder variants and the GMSK mapper, verified through layered testing to 41/41 encoder vectors and 0/10,000 decoder bit errors; and a measured on-board decoded throughput of 32.65 Mb/s with full latency and bottleneck characterization.

### 8.2 Research Contributions

The contributions claimed are those of a rigorous engineering case study: a documented staged pathway from ideal simulation to standardized-channel operation, exposing the composition defects (index domains, reference contamination) that isolated treatments omit; an empirical catalogue of HLS failure modes for stateful and control-dominated communication blocks, each with mechanism and remedy; a reproducible, division-free on-chip throughput measurement architecture; and quantified evidence on the HLS quality-of-result for a 64-state Viterbi decoder, including the agreement between scheduled and observed initiation intervals.

## 8.3 Lessons Learned

The condensed engineering lessons: increase channel realism in stages, or failures become unattributable; make group delay a first-class design quantity across every index conversion; never train an adaptive filter on a reference the channel did not produce and regenerate references with the transmitter's own operators; judge receivers by decoded errors, not surrogate metrics; treat HLS pragmas as architectural statements and co-simulation as non-optional; qualify every stream event on the complete handshake; expect and budget for defects in verification infrastructure; and manage encoder/decoder state assumptions explicitly at every block boundary.

## 8.4 Future Work

Future work divides into three strands. **Communication algorithms:** carrier-frequency-offset, timing-offset, phase-noise and quantization models (identified as the highest-value impairments); time-varying fading beyond block models; interleaving against fade bursts; normalized/adaptive preamble detection to suppress low-SNR false alarms; decision-directed adaptation after training; zero-tail termination in the framed system; and reducing the SNR required for BER  $10^{-5}$  over the HT channel below the present  $\approx 24$  dB. **Hardware:** deployment of the soft-decision decoder (II= 2 design complete) for its  $\approx 2$  dB gain; the II= 1 sliding-traceback `viterbi_k7` to lift the throughput ceiling; fixed-point characterization of the receiver chain; hardware realization of the remaining GMSK blocks, for which the mapper is already verified in C simulation; resolution of the IP-packaging toolchain fault; and FIFO-buffered pipeline studies. **Methodology:** hardware-in-the-loop BER testing with noise injection, cross-block streaming verification, and unknown-start-state robustness testing.

# Appendix A

## MATLAB Source Files

Table A.1: Principal MATLAB files by subsystem.

| File                                                                                  | Role                                                 |
|---------------------------------------------------------------------------------------|------------------------------------------------------|
| Waveform.m, TxBurst.m, FieldTypes.m                                                   | Frame definition, burst container, field enumeration |
| convEncoder.m                                                                         | Rate-1/2 $K=7$ encoder (System object)               |
| build_packet.m, build_multipacket.m                                                   | Burst and stream assembly                            |
| GaussianFilter.m, GmskMapper.m                                                        | Pulse shaping; phase-accumulator modulator           |
| GmskDemapper.m, RxDemodulator.m                                                       | Phase-derivative demodulation; symbol metrics        |
| detect_preamble.m, extract_payload.m                                                  | Synchronization; payload extraction                  |
| viterbi_soft_decoder.m                                                                | Sliding-window soft-decision decoder                 |
| channel_model.m, rayleigh_channel.m                                                   | GSM HT + AWGN; flat Rayleigh                         |
| estimate_channel_from_training.m, equalize_signal.m                                   | LS estimation; one-tap equalization                  |
| lms_equalizer_complex.m                                                               | Complex NLMS equalizer                               |
| gmsk_modulate_burst_ref.m, preamble_sample_indices.m                                  | Bit-exact training reference; preamble indices       |
| demod_aligned_burst.m                                                                 | Aligned-burst demodulation (no double trim)          |
| sweep_equalizer_params.m                                                              | $(\mu, L, D)$ sweep by byte errors                   |
| ideal_snr_vs_ber.m, rayleigh_ber_vs_snr.m, plot_ber_vs_snr.m, plot_ber_vs_snr_curve.m | BER campaign drivers                                 |
| read_axi_stream_csv.m, write_axi_stream_csv.m                                         | AXI-format CSV I/O                                   |

The complete MATLAB sources are publicly available in the project repository:

- Phase 01 (AWGN loop): <https://github.com/Dulina14/gmskModem/tree/main/gmsk-project-phase01>

- Phase 02 (GSM HT channel and NLMS equalization): <https://github.com/Dulina14/gmskModem/tree/main/gmsk-project-phase02>
- BER-versus-SNR experiments: <https://github.com/Dulina14/gmskModem/tree/main/gmsk-snr-vs-ber>

# Appendix B

## HLS Source Files

Table B.1: HLS and RTL-support sources.

| File(s)                                                                                        | Role                                                |
|------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| convEncoder_Seq.cpp,<br>convEncoder_Seq_tb.cpp,<br>hls_config.cfg                              | Encoder HLS source, testbench, configuration        |
| viterbi_decoder.h/.cpp, bmu.h/.cpp,<br>viterbi_decoder_tb.cpp                                  | 64-state hard-decision decoder and BER testbench    |
| viterbi_k7.h, viterbi_k7_tables.h,<br>viterbi_k7.cpp, viterbi_k7_tb.cpp,<br>viterbi_k7_hls.tcl | Parameterized decoder with constexpr trellis tables |
| convDecoder.h/.cpp,<br>convDecoder_tb.cpp                                                      | Soft-decision decoder ( $\eta=2$ )                  |
| viterbi_hls.h/.cpp, viterbi_tb.cpp,<br>viterbi_hls_project.tcl                                 | $K=3$ RTL-to-HLS conversion study                   |
| gmsk_mapper.hpp/.cpp, gmsk_lut.hpp,<br>gmsk_mapper_tb.cpp, generate_lut.py                     | GMSK mapper port and LUT generator                  |
| lfsr32.sv, throughput_bench_top.sv,<br>constraints.xdc                                         | Hardware measurement bench                          |
| tb_loopback_throughput.sv                                                                      | RTL loopback throughput testbench                   |

The complete HLS and hardware sources are publicly available in the project repository:

- HLS implementation (encoder, decoders, GMSK mapper, throughput bench): <https://github.com/Dulina14/gmskModem/tree/main/HLS-implementation>

# Appendix C

## RTL Diagrams

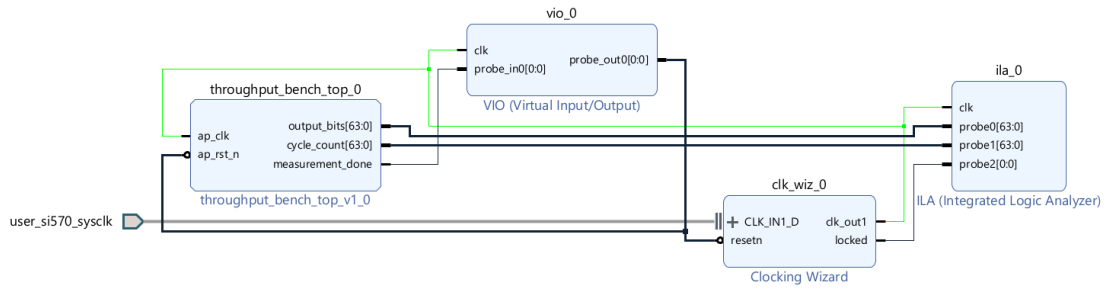


Figure C.1: Vivado block design of the complete GMSK modem system.

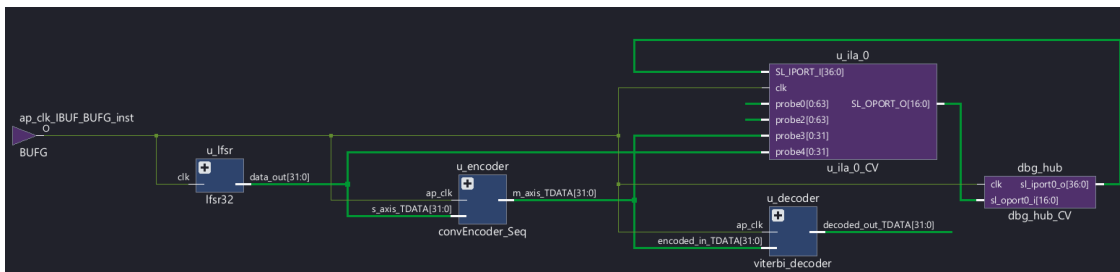


Figure C.2: Synthesized schematic of the `convEncoder_Seq` and `viterbi_decoder` modules.

# Appendix D

## Simulation Setup

RTL simulation used Vivado XSIM with `run -all` and VCD dumping; co-simulation used the Vitis HLS flow. Loopback parameters: 100 MHz clock (10 ns), 1000 input words, watchdog 500,000 cycles, optional `axis_fifo`. Hardware measurement: warmup 4096 cycles, window  $10^8$  cycles, ILA depth 1024, trigger `measurement_done == 1`. MATLAB BER runs: stopping rules as per Chapter 6.

Tool versions and reproducibility settings: all HLS synthesis and co-simulation used **Xilinx Vitis HLS 2025.2**, and all RTL simulation, synthesis, implementation and hardware debugging used **Xilinx Vivado 2025.2**, targeting the Digilent Zybo Z7-10 (Zynq-7000). Deterministic seeds were used throughout: `srand(42)` in the HLS decoder BER testbench, and `rng(12345)` for the waveform preamble/training sequence generation in MATLAB. The Vitis HLS Tcl flow for the parameterized decoder is provided as `viterbi_k7_hls.tcl` in the repository (Appendix B).

# Appendix E

## Experimental Data

The key measured data sets are reproduced below; complete raw logs (per-SNR bit and error counts, full equalizer sweep output, and Vivado report files) are available in the project repository referenced in Appendices A and B.

Table E.1: Equalizer development summary (GSM HT channel).

| Configuration                                                                   | Parameters                            | Payload result    |
|---------------------------------------------------------------------------------|---------------------------------------|-------------------|
| Best sweep result before root-cause fixes                                       | $\mu = 0.01$ , $L = 13$ ,<br>$D = 12$ | 31/34 byte errors |
| Final receiver (preamble training, group-delay-corrected extraction), 24 dB SNR | $\mu$ , $L$ , $D$ as tuned            | 0/34 byte errors  |

Table E.2: RTL loopback measurement (100 MHz).

| Quantity                             | Value                                             |
|--------------------------------------|---------------------------------------------------|
| Encoder input words                  | 1000 (32,000 information bits)                    |
| Encoder output words                 | 2000                                              |
| Decoder output words                 | 2000 (16 valid bits each)                         |
| First-input to first-output latency  | 84 cycles (840 ns)                                |
| Steady-state throughput              | $\approx 0.327$ bits/cycle ( $\approx 32.7$ Mb/s) |
| Observed decoder initiation interval | $\approx 48.9$ cycles/word (HLS report: 51)       |

Table E.3: On-board ILA counter readout (Zybo Z7-10, 50 MHz clock).

| Quantity                                     | Value                               |
|----------------------------------------------|-------------------------------------|
| Measurement window ( <code>cycs_q</code> )   | 100,000,000 cycles (2.0 s)          |
| Decoded bits counted ( <code>bits_q</code> ) | 65,306,144                          |
| Derived throughput                           | 32,653,072 b/s $\approx$ 32.65 Mb/s |
| Trigger condition                            | <code>measurement_done == 1</code>  |

# Bibliography

- [1] C. E. Shannon, “A mathematical theory of communication,” *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, Jul. 1948.
- [2] J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed. New York, NY, USA: McGraw-Hill, 2008.
- [3] B. Sklar, *Digital Communications: Fundamentals and Applications*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2001.
- [4] S. Haykin, *Communication Systems*, 4th ed. New York, NY, USA: Wiley, 2001.
- [5] J. B. Anderson, T. Aulin, and C.-E. Sundberg, *Digital Phase Modulation*. New York, NY, USA: Plenum Press, 1986.
- [6] S. Pasupathy, “Minimum shift keying: A spectrally efficient modulation,” *IEEE Commun. Mag.*, vol. 17, no. 4, pp. 14–22, Jul. 1979.
- [7] K. Murota and K. Hirade, “GMSK modulation for digital mobile radio telephony,” *IEEE Trans. Commun.*, vol. COM-29, no. 7, pp. 1044–1050, Jul. 1981.
- [8] P. Elias, “Coding for noisy channels,” *IRE Conv. Rec.*, vol. 3, pt. 4, pp. 37–46, 1955.
- [9] A. J. Viterbi, “Error bounds for convolutional codes and an asymptotically optimum decoding algorithm,” *IEEE Trans. Inf. Theory*, vol. IT-13, no. 2, pp. 260–269, Apr. 1967.
- [10] G. D. Forney, Jr., “The Viterbi algorithm,” *Proc. IEEE*, vol. 61, no. 3, pp. 268–278, Mar. 1973.
- [11] S. Lin and D. J. Costello, Jr., *Error Control Coding*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2004.
- [12] J. Hagenauer and P. Hoeher, “A Viterbi algorithm with soft-decision outputs and its applications,” in *Proc. IEEE GLOBECOM*, Dallas, TX, USA, 1989, pp. 1680–1686.

- [13] J. A. Heller and I. M. Jacobs, “Viterbi decoding for satellite and space communication,” *IEEE Trans. Commun. Technol.*, vol. COM-19, no. 5, pp. 835–848, Oct. 1971.
- [14] T. S. Rappaport, *Wireless Communications: Principles and Practice*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2002.
- [15] W. C. Jakes, Ed., *Microwave Mobile Communications*. New York, NY, USA: Wiley, 1974.
- [16] COST 207, *Digital Land Mobile Radio Communications: Final Report*. Luxembourg: Commission of the European Communities, 1989.
- [17] 3GPP, “GSM/EDGE Radio Access Network; Modulation,” 3rd Generation Partnership Project, Tech. Spec. TS 45.004.
- [18] 3GPP, “GSM/EDGE Radio Access Network; Radio transmission and reception,” 3rd Generation Partnership Project, Tech. Spec. TS 45.005.
- [19] S. Haykin, *Adaptive Filter Theory*, 4th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2002.
- [20] B. Widrow and S. D. Stearns, *Adaptive Signal Processing*. Englewood Cliffs, NJ, USA: Prentice Hall, 1985.
- [21] G. Fettweis and H. Meyr, “High-speed parallel Viterbi decoding: Algorithm and VLSI-architecture,” *IEEE Commun. Mag.*, vol. 29, no. 5, pp. 46–55, May 1991.
- [22] P. J. Black and T. H. Meng, “A 140-Mb/s, 32-state, radix-4 Viterbi decoder,” *IEEE J. Solid-State Circuits*, vol. 27, no. 12, pp. 1877–1885, Dec. 1992.
- [23] K. K. Parhi, *VLSI Digital Signal Processing Systems: Design and Implementation*. New York, NY, USA: Wiley, 1999.
- [24] U. Meyer-Baese, *Digital Signal Processing with Field Programmable Gate Arrays*, 4th ed. Berlin, Germany: Springer, 2014.
- [25] J. Cong *et al.*, “High-level synthesis for FPGAs: From prototyping to deployment,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 30, no. 4, pp. 473–491, Apr. 2011.
- [26] Xilinx, Inc., *Vitis High-Level Synthesis User Guide*, UG1399.
- [27] ARM Ltd., *AMBA 4 AXI4-Stream Protocol Specification*, ARM IHI 0051A, 2010.
- [28] Xilinx, Inc., *Viterbi Decoder LogiCORE IP Product Guide*, PG027.